Table of contents

A Study on Species in Our Datasets

> Here we will take a deeper look into our specific species. We will look at things like their breeding functions, eating habits, physiology, ecological preferences, and limitations.

Header

```
# ============================================================
# program:      species_study.ipynb
# purpose:      Analyze genus- and species-level fish abundance data to
identify
#               ecologically relevant taxa for further modeling. This
notebook
#               filters genera and species by abundance thresholds,
visualizes
#               distribution patterns, and compiles detailed ecological
profiles
#               for all focal species used in downstream analysis.
#
# usage:        Run interactively in Jupyter Notebook. Requires access to:
#                   - seth_genusCountData_2025.csv
#                   - seth_fishCountData_2025.csv
#                   - gov_envData10dts_2025.csv
#                   - gov_envData10dtsMetrics_2026.csv
```

```
#                   - seth_envData_2025.csv
#               All figures are generated inline; optional saving via
lpe().
#
# notes:        (1) Performs genus and species filtering using count
thresholds.
#               (2) Produces ordered barplots for genus and species
abundance.
#               (3) Includes curated ecological summaries for all focal
species.
#               (4) Designed as the documentation + exploratory analysis
stage
#                   preceding modeling notebooks.
#
# date:         06/24/2026
# programmer:   Korbin L. Brashears (KLB)
# ============================================================
```

> Importing required libraries, and creating global variables.

In [1]:
```
# Basic CSV and DataFrame Manipulation Libraries.

import pandas as pd                             # Pandas, for reading and manipulat
import numpy as np                              # Numpy, for more advanced manipula
import matplotlib.pyplot as plt                 # MatplotLib, for basic figures and
import matplotlib.patches as patches
import seaborn as sns                           # Seaborn, for more plotting and co


# Personal Plotting Functions

from functions import (
                        label_plot_edges as lpe,   # Label_plot_edges, a custom fun
                        scatter_trendLine_plotter, # Scatter_trendLine_plotter, plo

                        corr_decay_plotter
                        )                          # Corr_decay_plotter, computes a
                                                   # target attribute (Series) and
                                                   # features (DataFrame).

# Data Preprocessing Libraries

from sklearn.preprocessing import StandardScaler

# Dimensionality Reduction Libraries.

from sklearn.decomposition import PCA                   # PCA, is has functions to
from sklearn.model_selection import train_test_split                # Train_test_s

# Machine Learning Libraries

from xgboost import XGBClassifier, XGBRegressor         # XGBoost, is a gradient bo
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
```

```
from sklearn.metrics import (accuracy_score,
                             mean_squared_error,
                             r2_score,
                             classification_report,
                             confusion_matrix)          # Metrics, is a module that
```

> Reading the CSV files into pandas Data Frames.

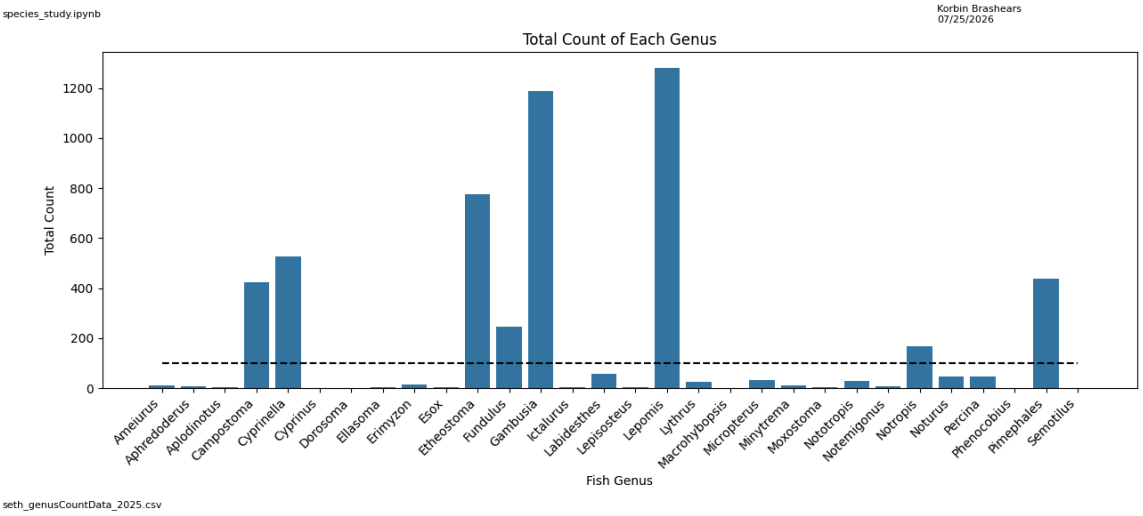
In [12]:
```
# ============================================== Defining all CSV Filenames ======

filenames = {
     "s_gcd" : "seth_genusCountData_2025.csv",
     "s_fcd" : "seth_fishCountData_2025.csv",
     "s_ed" : "seth_envData_2025.csv",
     "g_10dts" : "gov_envData10dts_2025.csv",
     "g_10dts_m" : "gov_envData10dtsMetrics_2026.csv",
}

# Using pandas' function "read_csv" to read our Genus and Species count CSV's into

s_gcd = pd.read_csv(f'../data/{filenames['s_gcd']}')     # Reading seth_genusCountD
s_fcd = pd.read_csv(f'../data/{filenames['s_fcd']}')      # Reading seth_fishCountD


# Using pandas' function "read_csv" to read our Environmental and Burn metric CSV's

s_ed = pd.read_csv(f'../data/{filenames['s_ed']}')                 # Reading seth_e
g_10dts = pd.read_csv(f'../data/{filenames['g_10dts']}')          # Reading gov_env
g_10dts_m = pd.read_csv(f'../data/{filenames['g_10dts_m']}')      # Reading gov_env

# ==================================== Put all DataFrames in a list for batch proc

datasets = [s_gcd, s_fcd, s_ed, g_10dts, g_10dts_m]


# ================================================ Clean all column names ======

for df in datasets:
    df.columns = (
        df.columns
        .str.strip()                            # remove leading/trailing spaces
        .str.replace(' ', '_')                  # replace internal spaces with unde
        .str.replace('\n', '', regex=False)     # remove newlines
    )

# ================================================ Declare Global Variables =====

FEATURE_NAMES = [
    "avg_Depth", "pf_Pool", "pf_Riffle", "pb_Mud", "pb_SmGravel",
```

```
    "pb_LgGravel", "avgMonthFlow_cfs", "CV_flow", "pf_Run",
    "pb_Boulder", "pb_Sand", "pft_Macrophytes", "pft_Wood",
    "maxMonthFlow_cfs", "minMonthFlow_cfs", "upstreamCumDA_km2",
    "slopePercent"
]
```

Getting Abundant Genus and Species

> Using a simple population threshold to first identify which of our genus have a strong enough presence for us to bother looking. Then we will move on to do the same to the species within those genus.

In [13]:
```
# ================================================ Variable Definition =========

genus_thresh = 100                                                    # Defining

# ================================================== Data Analysis =============

s_gcd_genus_cols = s_gcd.columns[2:]                                 # Selecting t
genus_sum = s_gcd[s_gcd_genus_cols].sum()                             # Summing e

# ================================================== Plotting ==================

plt.figure(figsize=(15, 6))                                           # Creating
sns.barplot(x=genus_sum.index, y=genus_sum.values)                    # Creating
plt.plot(range(0,len(genus_sum)),
         np.ones(len(genus_sum)) * genus_thresh,                      # Plotting
         color='black', linestyle="--")
plt.xticks(rotation=45, ha='right')                                   # Rotating
plt.xlabel('Fish Genus')                                              # Setting x
plt.ylabel('Total Count')                                             # Setting y
plt.title('Total Count of Each Genus')                                # Labeling

lpe(                                      # Calling the lpe to label the edges of t
    author_names="Korbin Brashears",
    data_filenames=filenames['s_gcd'],
    fig_filename="../figures/species_study/genus_barPlot.png",
    save = False                                                      # Change Fa
    )

plt.show()                                                            # Displayin
```



> From this we can select only the genus where there is enough fish in out dataset to be relevant to our research.
>
> We set genus_thresh to 100 to filter out all genus less than Notropis, at 167.

> Creating a Data Frame of only the Genus that pass our threshold and their count.

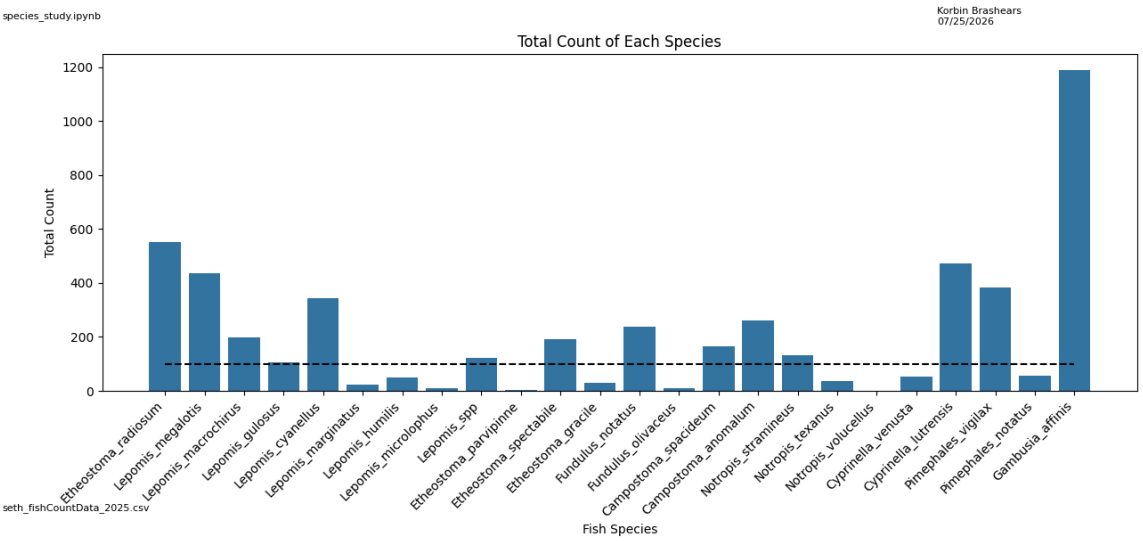
In [14]:
```
genus_sum_df = genus_sum.to_frame(name='Count').T                    # Converting our ge

rel_genus_cols = [gen for gen in genus_sum_df.columns if genus_sum_df[gen].iloc[0]
rel_genus_df = genus_sum_df[rel_genus_cols].T                        # Using the filtere

print(rel_genus_df)                                                  # Printing our new
```

```
           Count
Campostoma   424
Cyprinella   525
Etheostoma   777
Fundulus     247
Gambusia    1190
Lepomis     1281
Notropis     167
Pimephales   439
```

> Now we can repeat the same process to the Species within these Genus.

In [15]:
```
# ================================================ Variable Definition =========

species_thresh = 100                                                 # Defining
```

```
# ================================================ Data Analysis ================

rel_species_cols = [spec for spec in s_fcd.columns if spec.split("_")[0] in rel_gen

species_sum = s_fcd[rel_species_cols].sum()                                  # Summing e

# ================================================== Plotting ==================

plt.figure(figsize=(15, 6))                                                  # Creating
sns.barplot(x=species_sum.index, y=species_sum.values)                       # Creating
plt.plot(range(0,len(species_sum)),
         np.ones(len(species_sum)) * species_thresh,                         # Plotting
         color='black', linestyle="--")
plt.xticks(rotation=45, ha='right')                                          # rotating
plt.xlabel('Fish Species')                                                   # Setting x
plt.ylabel('Total Count')                                                    # Setting y
plt.title('Total Count of Each Species')                                     # Labeling

lpe(                                  # Calling the lpe to label the edges of the p
    author_names="Korbin Brashears",
    data_filenames=filenames['s_fcd'],
    fig_filename="../figures/species_study/species_barPlot.png",
    save = False                                                             # Change Fa
    )

plt.show()                                                                   # Displayin
```



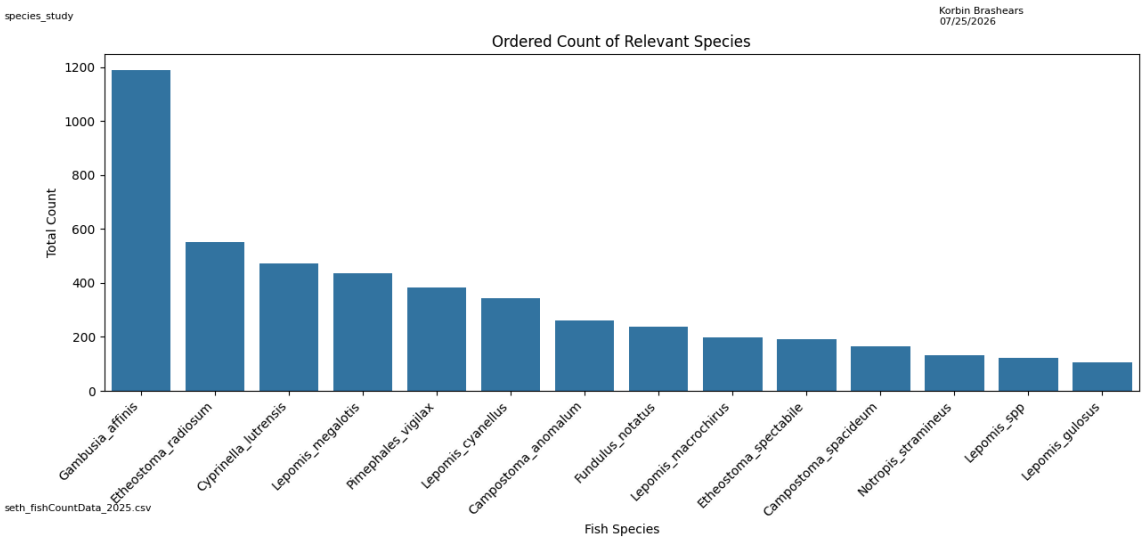
> Creating a Data Frame of only the Species that pass our threshold and their count.

In [16]:
```
species_sum_df = species_sum.to_frame(name='Count').T                # Converting ou

rel_species_cols = [gen for gen in species_sum_df.columns if species_sum_df[gen].il
rel_species_df = species_sum_df[rel_species_cols].T                    # Using the f

print(rel_species_df)                                                # Printing our ne
```

	Count
Etheostoma_radiosum	550
Lepomis_megalotis	435
Lepomis_macrochirus	198
Lepomis_gulosus	104
Lepomis_cyanellus	342
Lepomis_spp	122
Etheostoma_spectabile	190
Fundulus_notatus	237
Campostoma_spacideum	163
Campostoma_anomalum	261
Notropis_stramineus	132
Cyprinella_lutrensis	472
Pimephales_vigilax	384
Gambusia_affinis	1190

> Plotting the final species we are using in descending order.

In [17]:
```
# ================================================ Data Analysis ================

rel_species_df_ordered = rel_species_df.sort_values(by="Count", ascending=False)

# ================================================== Plotting ==================

plt.figure(figsize=(15, 6))                                          # Creating
sns.barplot(data=rel_species_df_ordered,
            x=rel_species_df_ordered.index,
            y="Count")                                               # Creating
plt.xticks(rotation=45, ha='right')                                  # Rotating
plt.xlabel('Fish Species')                                           # Setting x
plt.ylabel('Total Count')                                            # Setting y
plt.title('Ordered Count of Relevant Species')                       # Labeling

lpe(
    program_names="species_study",                                     # Calling
    author_names="Korbin Brashears",
    data_filenames=filenames['s_fcd'],
    fig_filename="../figures/species_study/genus_study_ordered_species_barPlot.png"
    save = False                                                     # Change Fa
    )

plt.show()                                                           # Displayin
```



Relevant Species Summary

> Now that we have out list of Species we want to focus our study on, we will do a deeper dive into each one.

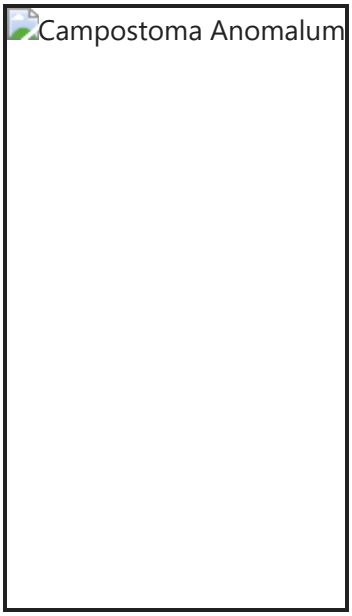
> **Relevant Genus and Species list:**
>
> **Campostoma:**
>
> Campostoma_spacideum
> Campostoma_anomalum
>
> **Cyprinella:**
>
> Cyprinella_lutrensis
>
> **Etheostoma:**
>
> Etheostoma_radiosum
> Etheostoma_spectabile
>
> **Fundulus:**
>
> Fundulus_notatus
>
> **Gambusia:**
>
> Gambusia_affinis
>
> **Lepomis:**
>
> Lepomis_megalotis
> Lepomis_macrochirus
> Lepomis_gulosus

> Lepomis_cyanellus
> Lepomis_spp
>
> **Notropis:**
>
> Notropis_stramineus
>
> **Pimephales:**
>
> Pimephales_vigilax

Campostoma

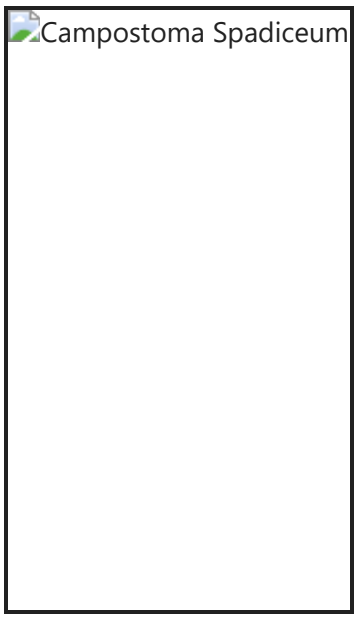
Campostoma anomalum (Central Stoneroller)

> **Eating Habits & Prey:** Primary herbivore. It acts as a stream "lawnmower," using a specialized, razor-sharp cartilaginous ridge on its lower jaw to scrape benthic algae, detritus, and diatoms off stones.
>
> **Migration & Other Habits:** Schooling, highly active, and non-migratory. It forms large grazing herds that move dynamically between riffles and pools.
>
> **Likes & Dislikes:** Likes rocky riffles with moderate to fast currents and high sunlight to fuel algae growth. Dislikes uniform sand or mud channels lacking hard structures.
>
> **Body Features:** Torpedo-shaped, brownish-tan body. It features an unusually long intestine wrapped completely around its swim bladder to aid in digesting tough plant matter.
>
> **Size Range (Weight):** Min: 0.01 lbs | Max: 0.22 lbs | Median: 0.07 lbs
>
> **Predators:** Largemouth bass, smallmouth bass, spotted bass, and larger sunfish species.
>
> **Ecological Concerns:** Secure and stable; they are critical for maintaining overall stream health by controlling nuisance algae blooms.
>
> **Species vs. Genus Notables:** It is globally widespread across North America, serving as the baseline ecological model for primary consumers within the genus, whereas its sister species are often highly localized geographic specialists.



Campostoma spadiceum (Highland Stoneroller)

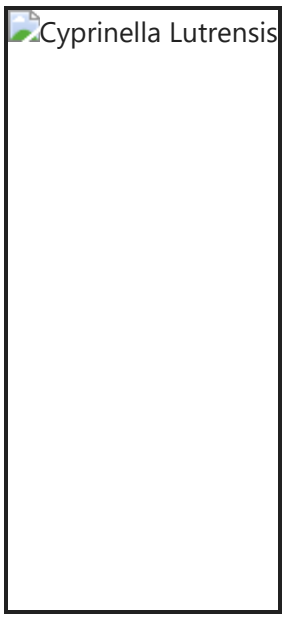
> **Eating Habits & Prey:** Herbivorous algivore. It targets periphyton, diatoms, and filamentous algae attached to hard river substrates.
>
> **Migration & Other Habits:** Schooling fish and non-migratory. Breeding males exhibit unique engineering behavior, using their heads to physically roll pebbles and dig out nesting trenches.
>
> **Likes & Dislikes:** Likes clear, cool, flowing upland streams with rocky or gravel substrates exposed to bright sunlight. Dislikes deep mud, heavy bank shade, and stagnant water.
>
> **Body Features:** Stout, cylindrical body. Breeding males develop vivid, fiery-red or orange fins year-round, a vertical black bar at the base of the tail, and hard, horn-like breeding tubercles on their heads.
>
> **Size Range (Weight):** Min: 0.01 lbs | Max: 0.18 lbs | Median: 0.06 lbs
>
> **Predators:** Centrachids, particularly bass, larger sunfish, and wading birds like herons.
>
> **Ecological Concerns:** Highly vulnerable to flash droughts and severe sedimentation that buries the clean rocky substrates required for feeding and nesting.
>
> **Species vs. Genus Notables:** Formally described as a distinct species in 2010, it differs from other stonerollers by maintaining its bright red fin pigmentation year-round and having a restricted range endemic to the Interior Highlands.



Cyprinella

Cyprinella lutrensis (Red Shiner)

> **Eating Habits & Prey:** Opportunistic omnivore. It feeds on aquatic insect larvae, terrestrial insects that fall into the water, algae, and occasionally the eggs or fry of competing fish species.
>
> **Migration & Other Habits:** Highly adaptable schooler and non-migratory. It broadcasts eggs over any available surface, including submerged roots, aquatic vegetation, or old nests of larger fish.
>
> **Likes & Dislikes:** Likes slow-moving pools, backwaters, and muddy, turbid channels. Dislikes pristine, cold, high-velocity mountain torrents.
>
> **Body Features:** Deep, highly compressed, diamond-shaped body. Breeding males turn a striking metallic blue with bright red-orange fins and a red crown.
>
> **Size Range (Weight):** Min: 0.002 lbs | Max: 0.015 lbs | Median: 0.005 lbs
>
> **Predators:** Channel catfish, flathead catfish, largemouth bass, gar, and herons.
>
> **Ecological Concerns:** Highly secure; its extreme resilience to poor water quality makes it a problematic invasive threat when introduced outside its native range.
>
> **Species vs. Genus Notables:** It is the most pollution-, silt-, and drought-tolerant member of the Cyprinella genus, making it exceptionally successful in heavily modified agricultural streams across Texas.

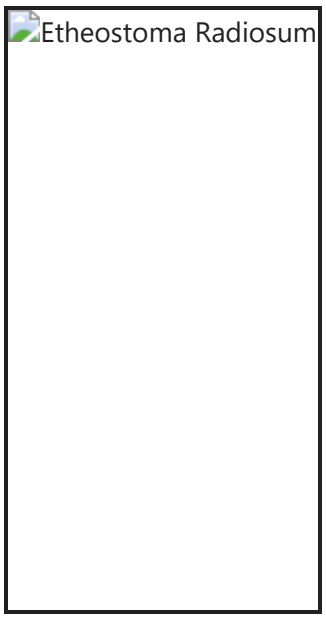


Etheostoma

Etheostoma radiosum (Orangebelly Darter)

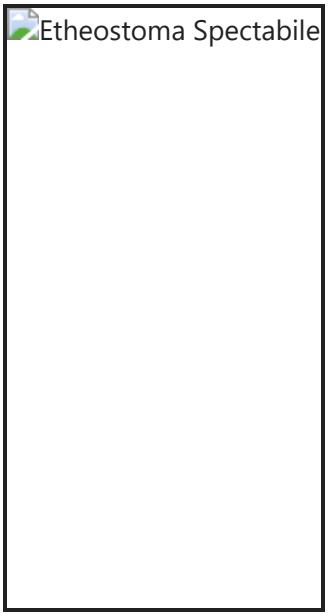
> **Eating Habits & Prey:** Specialized invertivore. It targets benthic aquatic insect larvae, primarily midges (Chironomidae), mayflies (Ephemeroptera), and caddisflies (Trichoptera).
>
> **Migration & Other Habits:** Strictly non-migratory. It remains in its localized stream reach. It spawns in early spring, depositing eggs directly into coarse gravel.
>
> **Likes & Dislikes:** Likes cool, clear headwaters and small streams with swift current velocities and gravel/rubble bottoms. Dislikes stagnant pools, high turbidity, and heavy mud or silt sedimentation.
>
> **Body Features:** Small, slender body with a blunt head and no swim bladder, keeping it tethered to the stream bottom. Breeding males display vibrant orange-red bellies and blue-green bands on their fins.
>
> **Size Range (Weight):** Min: 0.002 lbs | Max: 0.011 lbs | Median: 0.005 lbs
>
> **Predators:** Juvenile bass (Micropterus), larger sunfish (Lepomis), and wading birds.
>
> **Ecological Concerns:** Highly vulnerable to stream fragmentation (dams/culverts) and excessive siltation from land clearing, which smothers its gravel spawning beds.
>
> **Species vs. Genus Notables:** While the genus Etheostoma contains over 150 species across North America, E. radiosum has a highly localized geographic

> footprint, endemic primarily to the Red and Ouachita River drainages.



Etheostoma spectabile (Orangethroat Darter)

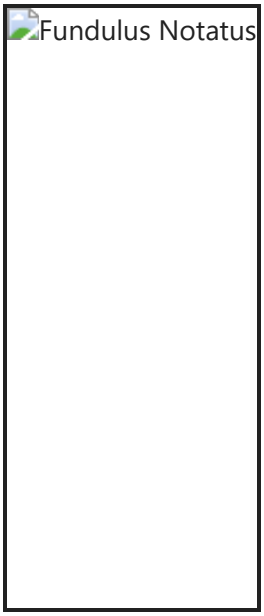
> **Eating Habits & Prey:** Benthic invertivore. It feeds on blackfly larvae, midge larvae, caddisfly larvae, and tiny amphipods within the rocky substrate.
>
> **Migration & Other Habits:** Non-migratory. Spawns in extremely shallow riffles during spring, burying its adhesive eggs in fine gravel beds.
>
> **Likes & Dislikes:** Likes small, shallow, sunlit creeks with moderate flow and mixed gravel/slab rock bottoms. Dislikes deep, muddy, or heavily choked slow rivers.
>
> **Body Features:** Small, colorful body. Breeding males develop a brilliant orange-red throat and branchiostegal membranes, with alternating blue and orange vertical bars down their sides.
>
> **Size Range (Weight):** Min: 0.002 lbs | Max: 0.010 lbs | Median: 0.004 lbs
>
> **Predators:** Sunfish, juvenile bass, and large predatory crayfish.
>
> **Ecological Concerns:** Vulnerable to urban and agricultural runoff; severe siltation glues gravel together, ruining its nesting and feeding spots.
>
> **Species vs. Genus Notables:** Compared to the highly sensitive E. radiosum, E. spectabile is noticeably more tolerant of intermittent flows and slight turbidity, enabling it to survive in smaller prairie streams.



Fundulus

Fundulus notatus (Blackstripe Topminnow)

> **Eating Habits & Prey:** Specialized surface feeder (invertivore). It eats terrestrial insects that fall into the water, mosquito larvae, and surface-skimming microcrustaceans.
>
> **Migration & Other Habits:** Non-migratory. It spends its life within the top two inches of the water column, laying eggs individually on submerged aquatic plants.
>
> **Likes & Dislikes:** Likes calm, slow-moving, or stagnant pools, sloughs, and stream margins with overhanging riparian vegetation. Dislikes swift currents and wide, open, deep water.
>
> **Body Features:** Slender, cigar-shaped body with a flattened head and an upturned (superior) mouth. A prominent, continuous black stripe runs from the tip of the snout to the tail.
>
> **Size Range (Weight):** Min: 0.003 lbs | Max: 0.014 lbs | Median: 0.006 lbs
>
> **Predators:** Sunfish, bass, kingfishers, and water snakes.
>
> **Ecological Concerns:** Secure, but highly dependent on healthy stream banks; removal of riparian trees directly destroys their shade and feeding zones.
>
> **Species vs. Genus Notables:** While most members of the Fundulus genus are coastal, estuarine, or salt-marsh specialists, F. notatus is uniquely adapted strictly to inland freshwater streams.

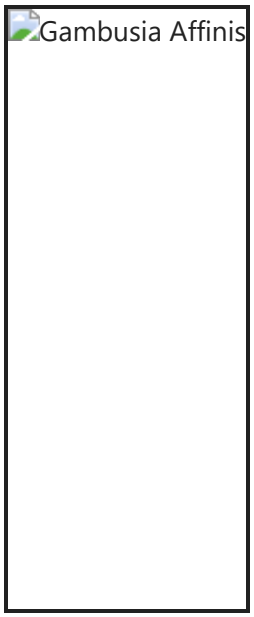


Gambusia

Gambusia affinis (Western Mosquitofish)

> **Eating Habits & Prey:** Surface insectivore. It is highly specialized in consuming mosquito larvae, alongside zooplankton and small surface-drifting insects.
>
> **Migration & Other Habits:** Surface-oriented schooler. Livebearer (viviparous)—females give birth to fully formed, swimming young rather than laying eggs, allowing rapid population explosions.
>
> **Likes & Dislikes:** Likes hot, completely stagnant, shallow shorelines and heavily weeded ponds. Dislikes fast, turbulent currents and cold water temperatures.
>
> **Body Features:** Tiny size, upturned mouth, and a rounded tail. Females are twice the size of males and display a dark, visible gravid spot near the vent when pregnant.
>
> **Size Range (Weight):** Min: 0.001 lbs | Max: 0.005 lbs | Median: 0.003 lbs
>
> **Predators:** Sunfish, juvenile bass, water snakes, and large predatory aquatic insects like diving beetles.
>
> **Ecological Concerns:** Secure. Highly invasive globally due to widespread stocking for mosquito control, though it is native to your East Texas study region.
>
> **Species vs. Genus Notables:** The definitive representative of the Gambusia genus, its live-bearing reproductive strategy is an evolutionary masterstroke

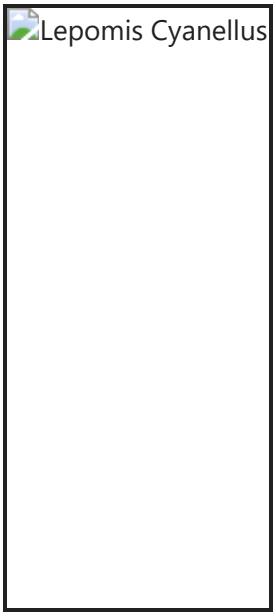
> for intermittent Texas creeks, completely bypassing the risk of eggs drying out or suffocating in mud during seasonal droughts.



Lepomis

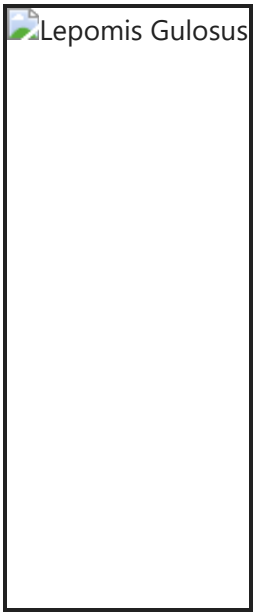
Lepomis cyanellus (Green Sunfish)

> **Eating Habits & Prey:** Aggressive, highly voracious carnivore. It eats anything it can fit in its mouth, including insects, crayfish, and smaller minnows.
>
> **Migration & Other Habits:** Exceptional pioneer species. It does not migrate long distances but quickly redistributes up dry creeks as soon as wet-season flows return.
>
> **Likes & Dislikes:** Tolerates almost anything. Likes small, stagnant pools and muddy ditches. Dislikes high-gradient, fast-flowing mountain streams.
>
> **Body Features:** More elongated and less deep-bodied than other sunfish. It has a large mouth, electric-blue flecks on its cheeks, and bright yellow margins on its dorsal and anal fins.
>
> **Size Range (Weight):** Min: 0.05 lbs | Max: 2.20 lbs | Median: 0.35 lbs
>
> **Predators:** Largemouth bass, flathead catfish, wading birds, and water snakes.
>
> **Ecological Concerns:** Secure; its extreme hardiness allows it to become an aggressive invasive risk when introduced outside its native basins.
>
> **Species vs. Genus Notables:** It is the most environmental-stress-tolerant member of the Lepomis genus, capable of surviving severe drought pools with low dissolved oxygen (DO) that kill all other sunfish.



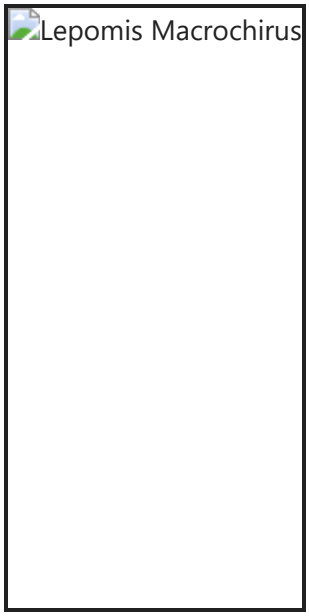
Lepomis gulosus (Warmouth)

> **Eating Habits & Prey:** Cryptic carnivore. It specializes in larger prey than most sunfish, actively hunting crayfish, minnows, and large dragonfly nymphs.
>
> **Migration & Other Habits:** Solitary and highly nocturnal or crepuscular. Non-migratory.
>
> **Likes & Dislikes:** Likes heavily vegetated, murky, low-flow pools, swamps, and bayous with soft organic debris or mud bottoms. Dislikes clear, fast-flowing gravel riffles.
>
> **Body Features:** Thick, robust body with a disproportionately large mouth (resembling a bass) and three to four dark stripes radiating backward from a red-ringed eye.
>
> **Size Range (Weight):** Min: 0.05 lbs | Max: 2.40 lbs | Median: 0.45 lbs
>
> **Predators:** Large bass, flathead catfish, and alligators.
>
> **Ecological Concerns:** Secure; exceptionally tolerant of hypoxic (low dissolved oxygen) environments and high water temperatures.
>
> **Species vs. Genus Notables:** Unlike the rest of the small-mouthed Lepomis genus, the Warmouth possesses an enlarged gape and a patch of small teeth on its tongue, making it ecologically behave more like a black bass.



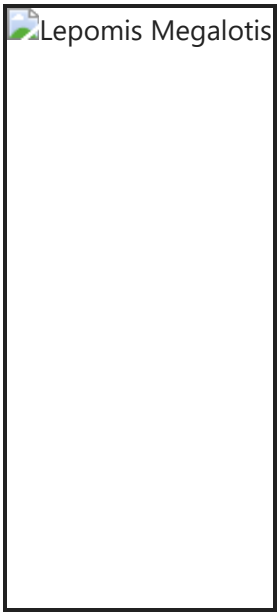
Lepomis macrochirus (Bluegill)

> **Eating Habits & Prey:** Generalist omnivore/invertivore. It consumes zooplankton, aquatic insect larvae, snails, worms, and small amounts of filamentous algae.
>
> **Migration & Other Habits:** Non-migratory. It forms dense schools and participates in synchronized colonial nesting where dozens of males pack nests closely together.
>
> **Likes & Dislikes:** Likes warm, quiet, well-vegetated pools, backwaters, and littoral zones. Dislikes high-velocity channels and completely bare, structured-less sand beds.
>
> **Body Features:** Deep, slab-sided body. It exhibits a dark spot at the posterior base of the soft dorsal fin and a solid black ear flap without light borders.
>
> **Size Range (Weight):** Min: 0.05 lbs | Max: 4.75 lbs | Median: 0.55 lbs
>
> **Predators:** Largemouth bass, longnose gar, snapping turtles, and blue herons.
>
> **Ecological Concerns:** Secure; often overpopulates and stunts in size if top-tier apex predators are removed from a stream system.
>
> **Species vs. Genus Notables:** It is the quintessential generalist of the Lepomis genus, possessing finer gill rakers than its peers, which allows it to filter tiny zooplankton far more efficiently than other sunfish.



Lepomis megalotis (Longear Sunfish)

> **Eating Habits & Prey:** Opportunistic invertivore and carnivore. It feeds on aquatic insects, small crustaceans, surface-drifted terrestrial insects, and small fish fry.
>
> **Migration & Other Habits:** Non-migratory. Males are highly territorial and colonial nesters, digging shallow, circular gravel depressions in bright sunlight.
>
> **Likes & Dislikes:** Likes clear, slow-moving streams, pools with physical structures (boulders, root wads, brush piles), and warm water. Dislikes swift currents, heavy mud bottoms, and deep shade.
>
> **Body Features:** Deep, laterally compressed body. It features a remarkably long, flexible, entirely black opercular (ear) flap bordered neatly in white or light blue.
>
> **Size Range (Weight):** Min: 0.04 lbs | Max: 1.74 lbs | Median: 0.25 lbs
>
> **Predators:** Largemouth bass, flathead catfish, herons, and otters.
>
> **Ecological Concerns:** Least concern; highly secure and adaptable.
>
> **Species vs. Genus Notables:** It stands out from the rest of the Lepomis genus due to its extreme, neon-bright blue and orange coloration and its elongated ear flap, making it far more colorful than close relatives like the Bluegill.



Lepomis spp (Unclassified Sunfish)

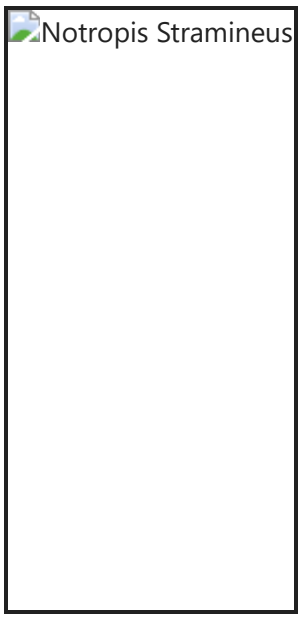
> **Eating Habits & Prey:** Generalist carnivores and invertivores. They consume a mix of aquatic insects, microcrustaceans, small fish, and surface drift depending on exact species composition.
>
> **Migration & Other Habits:** Primarily non-migratory, pool-dwelling fish. They display territorial nesting traits, with males fanning out depressions in sand or gravel substrates during late spring.
>
> **Likes & Dislikes:** Like slow-moving water, structural cover (woody debris, boulders), and warm stream temperatures. Dislike high-velocity channels and heavy siltation that ruins nesting beds.
>
> **Body Features:** Deep, laterally compressed bodies with spiny dorsal fins. This designation captures juvenile, hybrid, or unclassified sunfish specimens displaying intermediate traits.
>
> **Size Range (Weight):** Min: 0.04 lbs | Max: 3.00 lbs | Median: 0.35 lbs
>
> **Predators:** Larger black bass, catfish, wading birds, and water snakes.
>
> **Ecological Concerns:** Secure overall, though individual cryptic species or hybrids within the complex can reflect localized habitat degradation or displacement.
>
> **Species vs. Genus Notables:** Represents the collective ecological baseline of the genus Lepomis in Texas streams, capturing wide hybridization tendencies and multi-species visual overlaps in field surveys.

There is no image for this classification as Spp simply refers to a multitude of plural species that could not or were not identified as a specific species.

Notropis

Notropis stramineus (Sand Shiner)

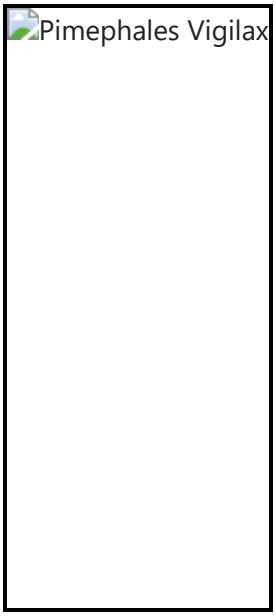
> **Eating Habits & Prey:** Generalist omnivore and detritivore. It eats small aquatic insects, terrestrial drift, algae, and organic bottom ooze.
>
> **Migration & Other Habits:** Schooling minnow and non-migratory. It broadcasts its adhesive eggs over open sand and gravel bars.
>
> **Likes & Dislikes:** Likes open, shallow, flowing prairie streams with loose, sandy or fine gravel bottoms. Dislikes dense aquatic weed beds and stagnant mud pools.
>
> **Body Features:** Small, translucent, silver-straw colored body with a dark stripe down the spine and a distinct line of dark dashes tracking its lateral line.
>
> **Size Range (Weight):** Min: 0.002 lbs | Max: 0.012 lbs | Median: 0.005 lbs
>
> **Predators:** Bass, sunfish, longnose gar, and terns.
>
> **Ecological Concerns:** Stable, but threatened by river channelization and upstream dams that stop the natural downstream transport of sand.
>
> **Species vs. Genus Notables:** The genus Notropis is vast and complex. N. stramineus is unique because it specializes in unstable, shifting sandy bottom habitats where most other shiners cannot successfully forage or spawn.



Pimephales

Pimephales vigilax (Bullhead Minnow)

> **Eating Habits & Prey:** Benthic omnivore and detritivore. It consumes bottom detritus, organic mud, diatoms, and tiny midge larvae.
>
> **Migration & Other Habits:** Schooling minnow. It exhibits advanced parental care: males choose the underside of flat rocks or logs, clean the surface, and aggressively guard the eggs.
>
> **Likes & Dislikes:** Likes sluggish pools, slow runs, and large river backwaters with sand, silt, or mud bottoms. Dislikes cold, fast-flowing, high-gradient mountain streams.
>
> **Body Features:** Blunt, rounded head with a heavy, pad-like snout. It has a dull silvery-tan body with a prominent black spot at the front base of the dorsal fin and another at the tail base.
>
> **Size Range (Weight):** Min: 0.002 lbs | Max: 0.016 lbs | Median: 0.006 lbs
>
> **Predators:** Immature catfish, bass, and wading birds.
>
> **Ecological Concerns:** Secure; handles agricultural runoff and high turbidity very well.
>
> **Species vs. Genus Notables:** During the breeding season, males develop a distinctive, thickened fleshy pad on the top of their head (more pronounced than in the related Fathead Minnow) used to physically clean and prep the underside of nesting rocks.



Driver Analysis And Prediction Models

Analysis Objectives & Methodological Framework

> **Using Fish Traits to Find Key Drivers:** Our main goal is to use the eating habits, body features, and preferences of each fish to understand why populations change. By connecting these traits directly to our stream data and climate data, we can figure out exactly what causes certain fish to thrive or disappear at different locations.

> **Testing Known Behaviors with Visuals:** We will use data charts and graphs to test if the fish are behaving the way textbooks say they should. For example, we can plot fish counts against mud levels (`pb_Mud`) or water depth (`avg_Depth`) to see if the field data actually backs up their known preferences.

> **Finding New Patterns & Climate Impacts:** Beyond testing what we already know, we want to look for surprising patterns or unusual fish behavior. By combining our local stream data with the 24-year satellite data analysis, from *time_series_analysis_2026.ipynb*, we can see how long-term problems—like repeated droughts (`palmer_z`) or fires—cause unexpected damage to the fish communities over time.

> **Building Predictive Models:** Finally, we will use everything we learn to build smart computer models. By knowing exactly which stream conditions and weather patterns matter most to each fish, we can create accurate models to predict where species will be present, where they will be absent, and which areas are at risk of a population crash.

Campostoma

Campostoma anomalum

Expectations & Hypotheses

- **Benthic Algae Scraping:** They use a hard jaw ridge to eat algae off rocks. We can test this by looking for higher fish counts in shallow areas with high sunlight exposure, which fuels algae growth.
- **Riffle & Substrate Dependence:** They require clean rocks to feed and roll pebbles for nests. We can test if they actively avoid muddy bottoms and prefer areas with fast-moving, shallow water over gravel.
- **Flow & Weather Sensitivity:** They live in shallow habitats that dry out quickly. We can test if their populations drop at sites with low minimum stream flows or extended periods of high summer heat.
- **Community Schooling:** They move in large grazing herds. We can test this by checking if their presence overlaps with other small, rock-dwelling species or if they are excluded from deep water by larger predators.

- **Food & Habitat Availability:** Fish counts will show a strong upward curve in shallow, rocky stream runs, but will flatline or drop to zero as the stream bottom switches from gravel to thick mud where algae cannot grow.
- **Drought & Flow Dynamics:** Extended regional droughts and highly unstable, flashy stream flows will trigger severe population crashes by drying out shallow riffle zones and trapping fish in stagnant pools.
- **Farming & Siltation Stress:** High agricultural land use upstream will negatively impact fish counts because soil erosion creates muddy bottoms that bury rocky feeding grounds and cause dangerous drops in dissolved oxygen.
- **Predator & Cohabitant Relationships:** Stoneroller populations will positively overlap with small benthic darters that share the same rocky riffles due to shared habitat needs, but will be absent from deep pools with high predator counts.

> **Expected Drivers We Will Observe:**

> **Hypotheses:**

> **Known Behavior & Expectations:**

- **Food & Habitat Metrics:** Strong positive correlations with rocky bottom types (`pb_SmGravel` , `pb_LgGravel` , `pb_Cobble`) and fast stream sections (`pf_Run` , `pf_Riffle`). Strong negative correlations with deep pools (`avg_Depth` , `pf_Pool`) and mud (`pb_Mud`).
- **Drought & Flow Metrics:** Sharp population drops when long-term drought indices (`palmer_z` , `palmer_p`) stay negative, when summer heatwaves (`maxT`) spike in the

time series, or when minimum flows (`minMonthFlow_cfs`) approach zero. High flow variability (`CV_flow`) will also lower presence.

- **Farming & Siltation Metrics:** A clear drop-off in fish counts at sites where upstream farming signals (`alfalfa` , `grass`) cause mud levels (`pb_Mud`) to rise and dissolved oxygen (`DO`) to fall.
- **Predator & Cohabitant Metrics:** High positive spatial overlap with *Etheostoma* species counts, and clear negative spatial relationships with larger predatory sunfish (`Lepomis_gulosus`) and bass (`Micropterus`).

> The Central Stoneroller (*Campostoma anomalum*) is an obligate benthic algivore, so its abundance is expected to depend heavily on clean, stable gravel substrates that support periphyton growth. Because benthic algae require both sunlight and sediment-free surfaces, we expect C. anomalum to be strongly associated with riffles that receive high solar radiation.

> Due to its streamlined body and schooling behavior, C. anomalum prefers rocky riffle habitats over slow, depositional pools. Moderate to fast flow helps maintain these habitats by flushing fine sediments downstream, preventing the smothering of algal resources.

> During the breeding season, males excavate gravel to build communal spawning pits. For this reason, we expect the species to prefer small, movable gravel that can be manipulated during nest construction.

> As a small schooling species, C. anomalum is likely to avoid areas with high densities of larger predatory or aggressive fish (e.g., *Lepomis* spp.). We will test whether its abundance is inversely related to these larger competitors.

> Conversely, we expect positive associations with other small schooling minnows that share similar flow and habitat preferences.

> **From this we can assume their population will be related to the features as follows:**

- Positively related to **pb_SmGravel** (ideal substrate for algae and nest building).
- Inversely related to **pb_Mud**, **pb_Sand**, and **pb_Bedrock** (fine sediments smother algae; bedrock lacks loose gravel).
- Positively related to the various **sunshine** metrics (sunlight drives periphyton productivity).
- Inversely related to **pft_Macrophytes** (macrophytes shade the substrate and outcompete benthic algae).

- Possibly inversely related to **pft_Wood** (floating wood may block sunlight reaching the stream bed).

- **Flow habitat expectations:**

  - Positively related to **pf_Riffle** (preferred high-velocity, rocky habitat).
  - Positively related to **pf_Pool** (may reflect transitional or mixed habitats where riffles and pools co-occur).
  - Inversely related to **pf_Run** (runs may represent more uniform, less structured habitat).
  - However, because these flow metrics can be subjective or inconsistently interpreted in field surveys, **all three will be tested empirically**.
- Inversely related to **local agriculture** if a metric is available, due to siltation and turbidity degrading riffle habitats.

> Many of these are directional hypotheses based on ecological expectations; the dataset will ultimately determine which relationships hold true.

> **First we will simply create a correlation decay plot, with the following metrics, to get a general idea of our metrics' relation to C. Anomalum:**

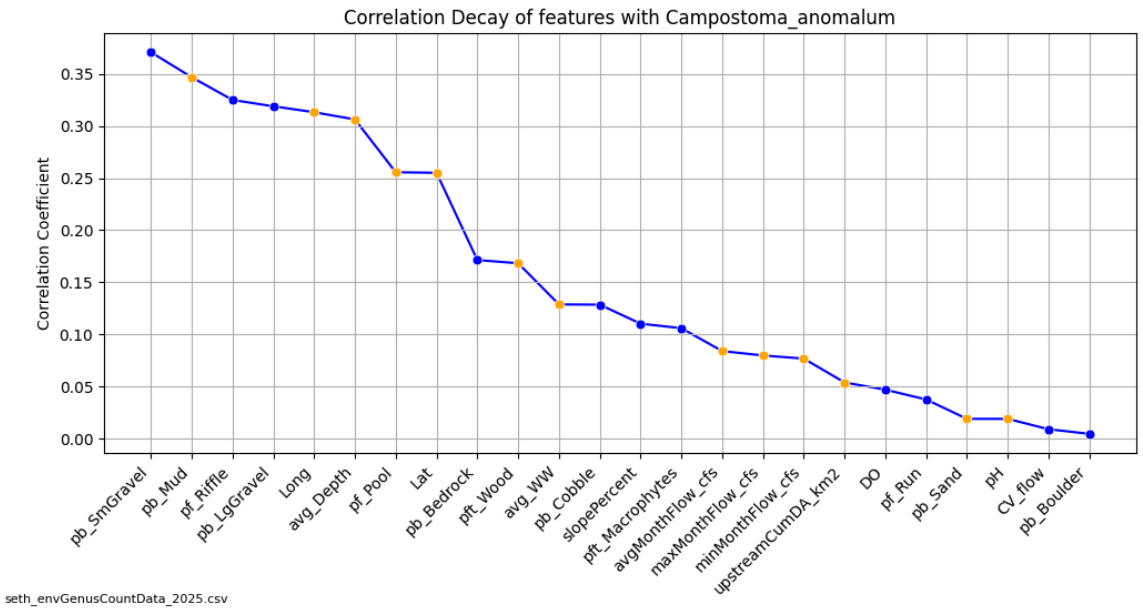
- The environmental metrics
- The burn / condensed time series metrics
- The expected key drives / metrics of interest

In [18]:
```
# All environmental metrics
env_m = s_ed.columns[1:]

# All burn metrics
burn_m = g_10dts_m.columns[1:]

# EKD environmental metrics (manually selected)
ekd_env_m = [
    "pb_SmGravel",
    "pb_Mud",
    "pb_Sand",
    "pb_Bedrock",
    "pft_Macrophytes",
    "pft_Wood",
    "pf_Riffle",
    "pf_Pool",
    "pf_Run"
]

# Suffixes we want to keep from burn metrics
ekd_suffixes = ["_mean"]

# Select only burn metrics ending with these suffixes
ekd_burn_m = [col for col in g_10dts_m.columns if any(col.endswith(suf) for suf in
```

```
# Combine into one EKD DataFrame
ekd_m = pd.concat([s_ed[ekd_env_m], g_10dts_m[ekd_burn_m]], axis=1)

# Run correlation-decay plots
corr_decay_plotter(attribute=s_fcd['Campostoma_anomalum'], features=s_ed[env_m])
corr_decay_plotter(attribute=s_fcd['Campostoma_anomalum'], features=g_10dts_m[ekd_b
corr_decay_plotter(attribute=s_fcd['Campostoma_anomalum'], features=ekd_m)
```

functions.py

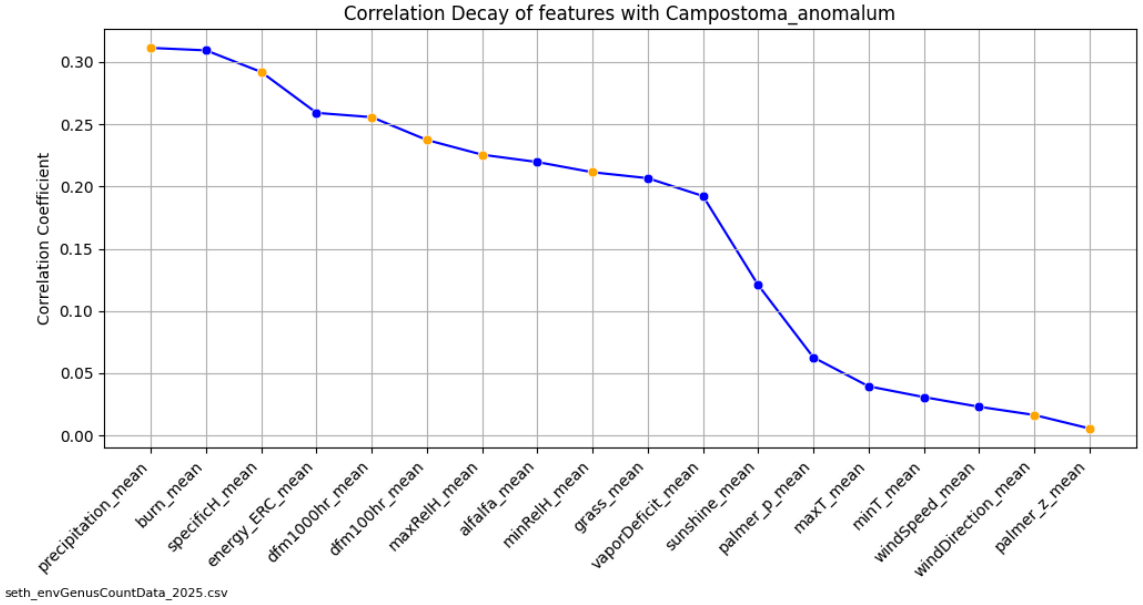
Korbin Brashears
07/25/2026



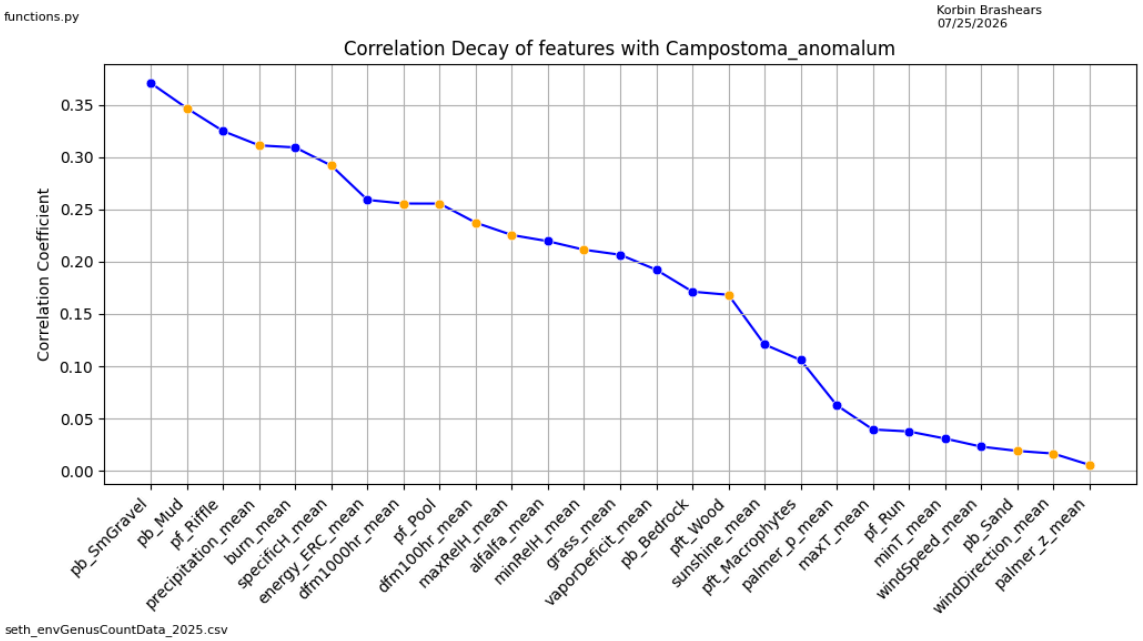
seth_envGenusCountData_2025.csv

functions.py

Korbin Brashears
07/25/2026

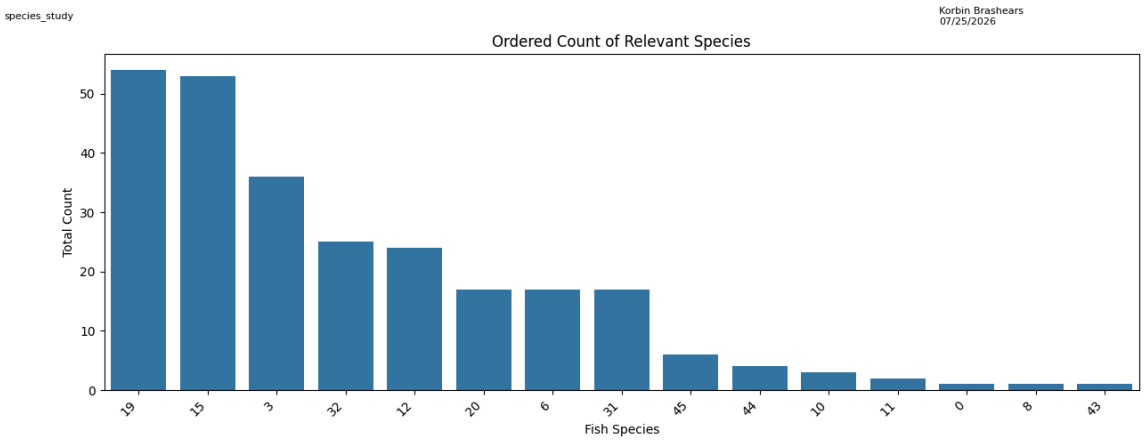


seth_envGenusCountData_2025.csv



> **Next we will run a principle component analysis to the sites containing C. Anomalum to find out which metrics explain the most variance within those sites:**

In [19]:
```
c_anomalum_sites = s_fcd[s_fcd['Campostoma_anomalum'] > 0]

# ================================================ Data Analysis ================

c_anomalum_sites_ordered = c_anomalum_sites.sort_values(by="Campostoma_anomalum", a

# ================================================== Plotting ==================

plt.figure(figsize=(15, 6))                                          # Creating
sns.barplot(data=c_anomalum_sites_ordered,
            x=c_anomalum_sites_ordered.index,
            y="Campostoma_anomalum",
            order=c_anomalum_sites_ordered.index)
plt.xticks(rotation=45, ha='right')                                  # Rotating
plt.xlabel('Fish Species')                                           # Setting x
plt.ylabel('Total Count')                                            # Setting y
plt.title('Ordered Count of Relevant Species')                       # Labeling

lpe(
    program_names="species_study",                                     # Calling
    author_names="Korbin Brashears",
    data_filenames=filenames['s_fcd'],
    fig_filename="../figures/species_study/genus_study_ordered_species_barPlot.png"
    save = False                                                     # Change Fa
    )

plt.show()                                                           # Displayin

print(c_anomalum_sites_ordered['Campostoma_anomalum'])
```



seth_fishCountData_2025.csv

```
19    54
15    53
3     36
32    25
12    24
20    17
6     17
31    17
45     6
44     4
10     3
11     2
0      1
8      1
43     1
Name: Campostoma_anomalum, dtype: int64
```

> Several sites contain fewer than 10 *Campostoma anomalum*.
> Because this species is strongly schooling, such low counts are not ecologically representative.
> For all PCA and niche-space analyses, we therefore restrict to sites with **N > 10** individuals.

In [20]:
```
c_anomalum_sites = s_fcd[s_fcd['Campostoma_anomalum'] > 10]

print(c_anomalum_sites['Campostoma_anomalum'])
```

```
3     36
6     17
12    24
15    53
19    54
20    17
31    17
32    25
Name: Campostoma_anomalum, dtype: int64
```

In [31]:
```
# Extract the site indices
idx = c_anomalum_sites.index

# Filter environmental and burn metrics to these sites
env_filtered = s_ed.drop(['SiteN', 'State'], axis=1).loc[idx]
#burn_filtered = g_10dts_m.drop('SiteN', axis=1).loc[idx]

# Combine all environmental + burn metrics
metrics_all = s_ed.drop(['SiteN', 'State'], axis=1)
metrics_filtered = env_filtered

# Combine all environmental + burn metrics
# metrics_all = pd.concat([s_ed.drop('SiteN', axis=1), g_10dts_m.drop('SiteN', axis
# metrics_filtered = pd.concat([env_filtered, burn_filtered], axis=1)
```

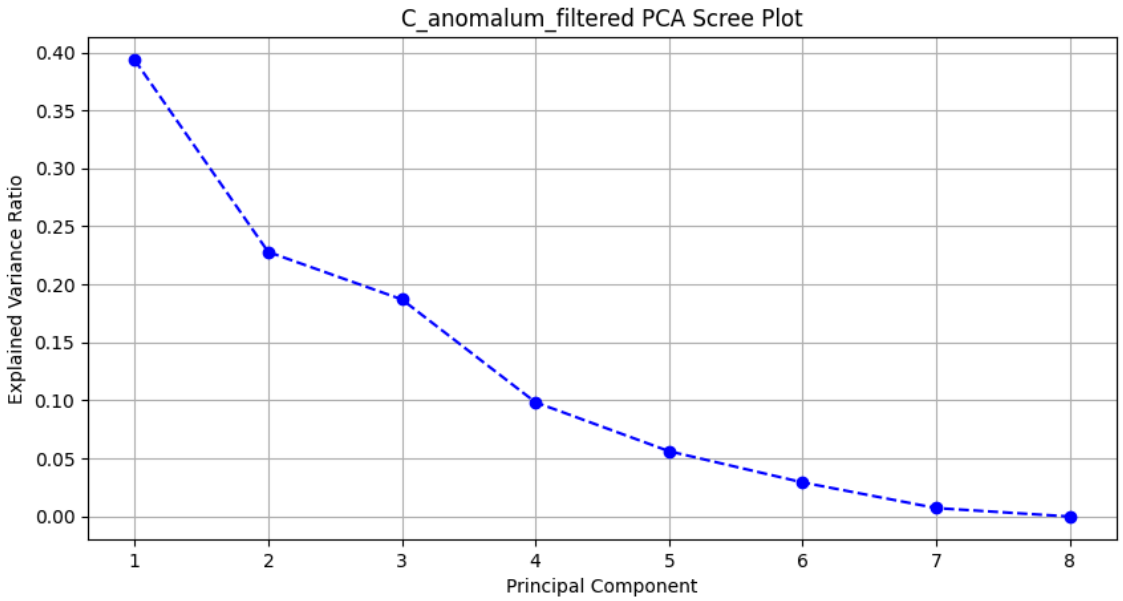
In [25]:
```
def pca_decay_plotter(features, title_prefix="", save=False):
    # Standardize
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(features)

    # PCA
    pca = PCA()
    pca.fit(X_scaled)

    explained = pca.explained_variance_ratio_
    components = np.arange(1, len(explained) + 1)

    # Scree plot
    plt.figure(figsize=(10, 6))
    plt.plot(components, explained, marker='o', linestyle='--', color='blue')
    plt.xticks(components)
    plt.xlabel("Principal Component")
    plt.ylabel("Explained Variance Ratio")
    plt.title(f"{title_prefix} PCA Scree Plot")
    plt.grid(True)

    lpe(
        author_names="Korbin Brashears",
        data_filenames=[filenames['s_ed'], filenames['s_fcd']],
        fig_filename=f"../figures/pca_{title_prefix}_scree_plot.png",
        save=save
    )

    plt.show()

    # Loadings
    loadings = pd.DataFrame(
        pca.components_.T,
        index=features.columns,
        columns=[f"PC{i+1}" for i in range(len(explained))]
    )

    print("\n====================================================")
    print(f"PCA Loadings for {title_prefix}")
    print("====================================================")
```

```
    print(loadings.round(4))

    return loadings
```

In [32]:
```
pca_loadings = pca_decay_plotter(metrics_filtered, title_prefix="C_anomalum_filtere
```

species_study.ipynb

Korbin Brashears
07/25/2026

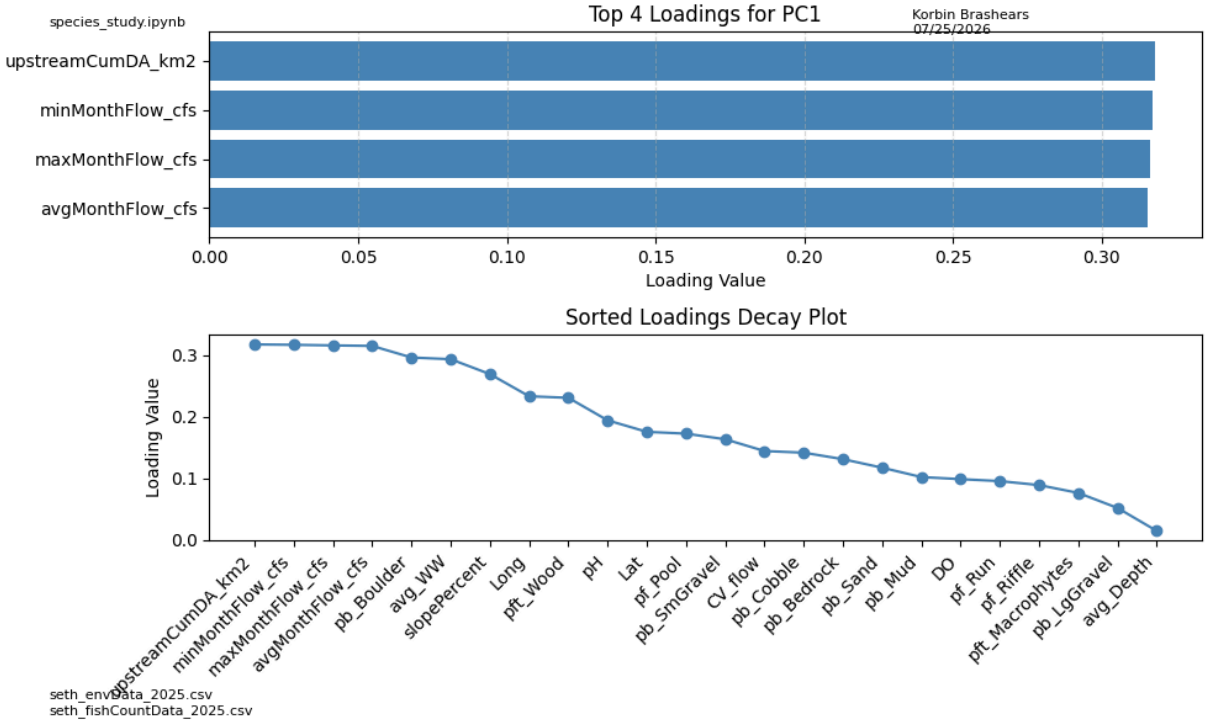


seth_envData_2025.csv
seth_fishCountData_2025.csv

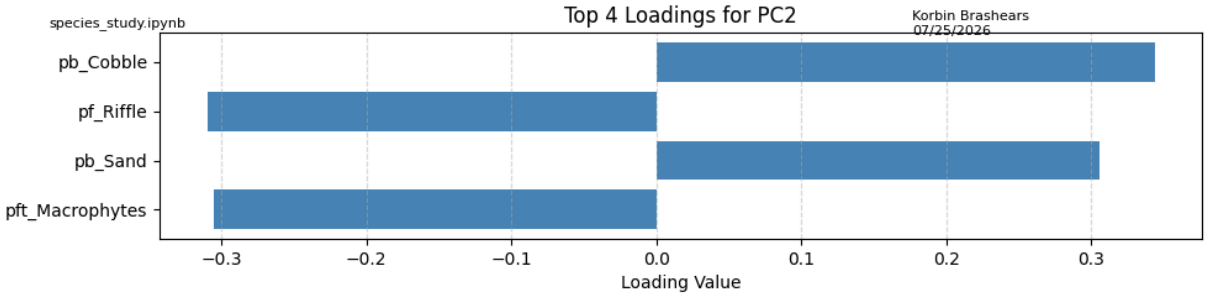
```
==================================================
PCA Loadings for C_anomalum_filtered
==================================================
                       PC1     PC2     PC3     PC4     PC5     PC6     PC7  \
Lat                -0.1759  0.1539 -0.3328 -0.1014 -0.0664  0.2123 -0.3288
Long                0.2336  0.1027  0.1873 -0.1663 -0.3731 -0.0260  0.3010
DO                 -0.0990  0.2642  0.0647 -0.3881  0.3265 -0.0663  0.1404
pH                  0.1945  0.1398 -0.3306 -0.0668 -0.1100 -0.0764 -0.2798
avg_WW              0.2938  0.1443  0.0402  0.0232  0.0575 -0.2861  0.0184
avg_Depth          -0.0149  0.2776 -0.3275 -0.0974 -0.0658 -0.2599  0.3277
pf_Pool            -0.1729 -0.1580 -0.3399  0.1198  0.0411  0.1809  0.1282
pf_Run              0.0958  0.2583  0.3471  0.0098 -0.0393  0.0252 -0.1601
pf_Riffle           0.0894 -0.3100 -0.1885 -0.2369  0.0166 -0.3814  0.1381
pb_Mud              0.1024 -0.2903 -0.2291 -0.2917  0.0237  0.0492 -0.0543
pb_Sand             0.1176  0.3057 -0.0577  0.2716  0.3018  0.2464 -0.1270
pb_SmGravel         0.1638  0.1170 -0.3114  0.3011  0.0128 -0.1623  0.1382
pb_LgGravel         0.0522  0.2580 -0.0476  0.4769 -0.1856 -0.1494 -0.0032
pb_Cobble          -0.1420  0.3438 -0.0273 -0.1080  0.2658  0.1889  0.2534
pb_Boulder         -0.2966 -0.0689 -0.0799  0.0063 -0.2566  0.1835  0.1031
pb_Bedrock         -0.1314 -0.0345  0.4240 -0.0825 -0.0386 -0.0643 -0.1038
pft_Macrophytes    -0.0766 -0.3055  0.0823  0.3836 -0.2021  0.0119  0.0518
pft_Wood            0.2311 -0.2012 -0.0159  0.0300  0.2101  0.5324  0.2540
upstreamCumDA_km2   0.3178  0.0100 -0.0113 -0.0616 -0.1178  0.1562  0.0164
slopePercent       -0.2696  0.1621  0.0374  0.0213 -0.2839  0.1162  0.5026
avgMonthFlow_cfs    0.3154  0.0263  0.0028 -0.0694 -0.1434  0.1544  0.0745
CV_flow             0.1447 -0.2169  0.0978  0.2567  0.4794 -0.1756  0.2844
maxMonthFlow_cfs    0.3162  0.0224 -0.0003 -0.0659 -0.1359  0.1557  0.0587
minMonthFlow_cfs    0.3172  0.0127 -0.0089 -0.0599 -0.1179  0.1735  0.0257

                       PC8
Lat                 0.2392
Long               -0.3816
DO                  0.1129
pH                  0.2100
avg_WW              0.5378
avg_Depth          -0.1765
pf_Pool            -0.2074
pf_Run              0.0216
pf_Riffle           0.1370
pb_Mud              0.0053
pb_Sand            -0.0455
pb_SmGravel        -0.0740
pb_LgGravel        -0.0690
pb_Cobble           0.0559
pb_Boulder          0.1523
pb_Bedrock          0.0549
pft_Macrophytes     0.2763
pft_Wood            0.0711
upstreamCumDA_km2   0.0576
slopePercent        0.4195
avgMonthFlow_cfs   -0.0832
CV_flow             0.0878
maxMonthFlow_cfs    0.1837
minMonthFlow_cfs    0.0836
```

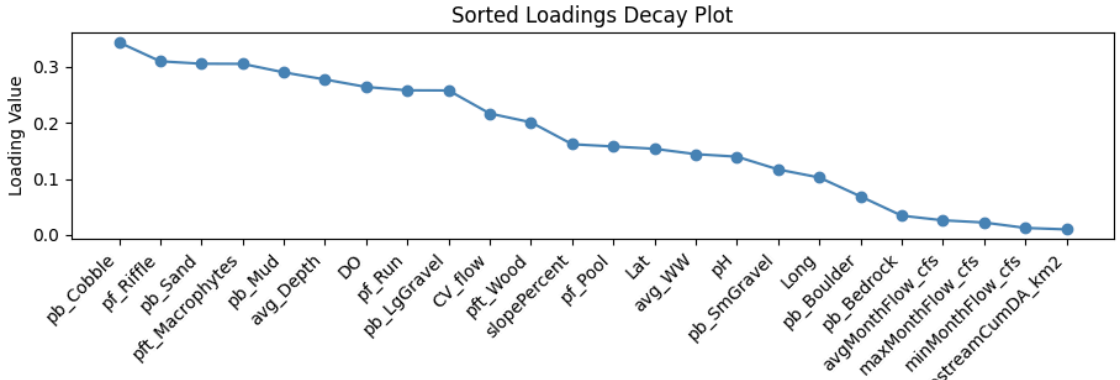
In [27]:
```
def plot_top_pca_loadings(loadings, top_pcs=3, top_vars=5):
    """
    Visualize the top N loadings for the top N principal components.
    Produces one horizontal bar chart per PC.
    """

    for pc in range(top_pcs):
        pc_name = f"PC{pc+1}"

        # Get absolute loadings and sort
        sorted_loadings = loadings[pc_name].abs().sort_values(ascending=False)

        # Select top variables
        top_loads = sorted_loadings.head(top_vars)
        top_vars_names = top_loads.index
        top_vals = loadings.loc[top_vars_names, pc_name]

        # Plot
        plt.figure(figsize=(10, 6))

        plt.subplot(2, 1, 1)

        plt.barh(top_vars_names, top_vals, color="steelblue")
        plt.xlabel("Loading Value")
        plt.title(f"Top {top_vars} Loadings for {pc_name}")
        plt.gca().invert_yaxis()  # highest at top
        plt.grid(axis="x", linestyle="--", alpha=0.5)

        plt.subplot(2, 1, 2)

        plt.plot(
            sorted_loadings.index,
            sorted_loadings.values,
            marker='o',
            color="steelblue"
        )

        plt.xticks(sorted_loadings.index, rotation=45, ha='right')
        plt.ylabel("Loading Value")
        plt.title("Sorted Loadings Decay Plot")

        plt.tight_layout()

        lpe(
            author_names="Korbin Brashears",
            data_filenames=[filenames['s_ed'], filenames['s_fcd']],
            fig_filename=f"../figures/pca_top_loadings_{pc_name}.png",
            save=False
        )

        plt.show()
```

In [28]:
```
plot_top_pca_loadings(pca_loadings, top_pcs=3, top_vars=4)
```

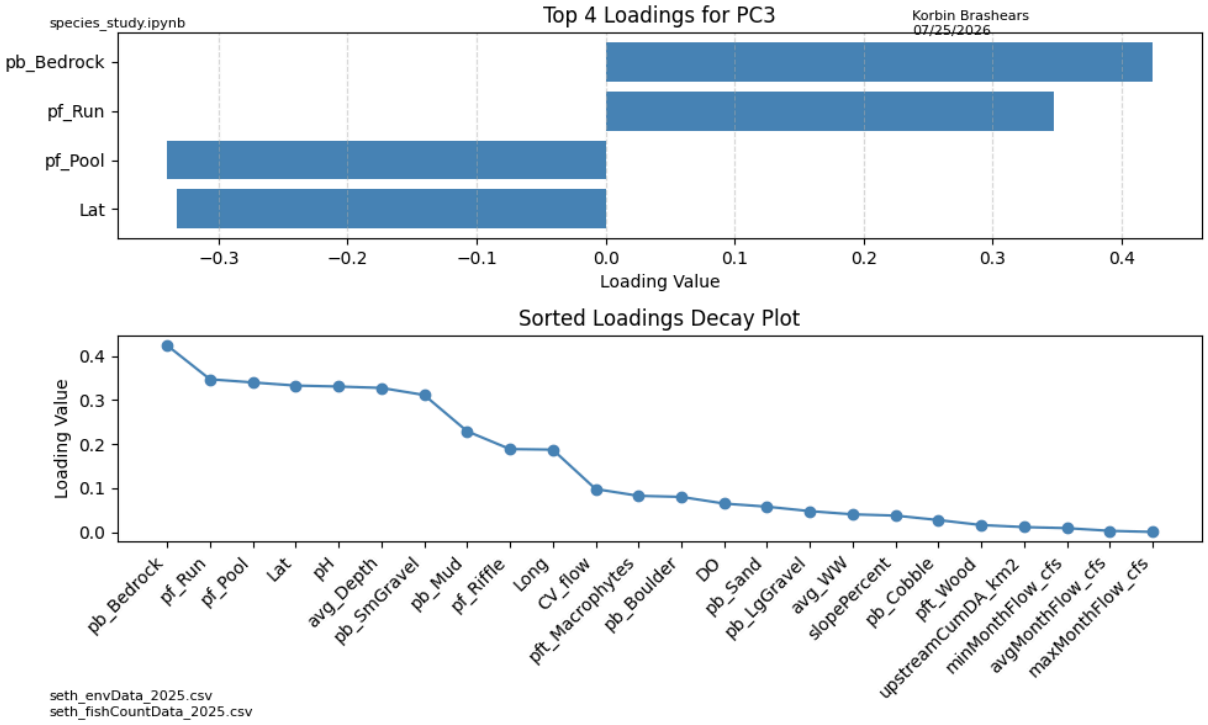


seth_envData_2025.csv
seth_fishCountData_2025.csv





seth_envData_2025.csv
seth_fishCountData_2025.csv



Current interpretation:

- PC1, PC2, and PC3 explain a relevant level of variance in the sites containing N >= 10 of our *C. anomalum*.
- This suggests that the loadings of these principal components may point to key drivers of the population.

Principal Component Highest Loadings

- **PC1:**
  - Loadings seem to point to the min, max, average, and upstream flow metrics primarily contributing to this component.
  - PC1 has a shallow drop-off, but its top four loadings show that the flow metrics contribute the most to the variance.
- **PC2:**
  - Loadings seem to point to the Cobble, Riffle, Mud, and Sand metrics primarily contributing to this component.
  - At first it seems odd that mud, cobble, and sand load highly alongside riffle, but using the correlation heat matrix from `kbrashears_fishData_report_02042026.ipynb`, these metrics actually show relatively high correlation (around 0.3).
  - This likely reflects that the bottom composition of the river has a strong relationship with flow and channel shape, similar to riffle.
- **PC3:**

- Loadings seem to point to the Bedrock, Depth, Pool, and Run metrics primarily contributing to this component.
- This again indicates that pooling/flow combined with bottom composition explains much of the variance in our sites containing *C. anomalum*.

> **Conclusion**
> These results support that the most influential variables in our clustering of these sites are the flow characteristics and the composition of the streambed. This suggests that riverbed composition may be the most telling factor for *C. anomalum*, as flow often correlates with whether the substrate contains the small gravel needed for spawning.
>
> In areas with high sediment but strong flow, larger gravel can settle while mud and sand wash away; in low-flow areas, mud and sand accumulate, potentially smothering eggs or preventing spawning altogether.
>
> The primary exception is bedrock bottoms: when the substrate is solid rock, flow does not separate gravel or sediment. In these cases, the water may be fast or slow, but flow has little effect on substrate composition because the bottom cannot shift.

In [ ]:

Niche Variance Interpretation

> To understand which environmental metrics define the realized niche of *Campostoma anomalum*, we computed the variance of each metric across only the sites where the species occurs in ecologically meaningful numbers (N > 10).
>
> Metrics with **low variance** are highly consistent across all anomalum sites, indicating **key habitat requirements**. These variables do not vary much within occupied habitat and therefore represent stable environmental conditions preferred by the species.
>
> Metrics with **high variance** fluctuate widely even where anomalum is present, suggesting that the species **tolerates** variation in these conditions. These variables are less likely to be primary drivers of habitat selection.
>
> This variance-based approach complements PCA by identifying which environmental factors are stable within the species' niche (important) versus those that vary substantially (tolerated).

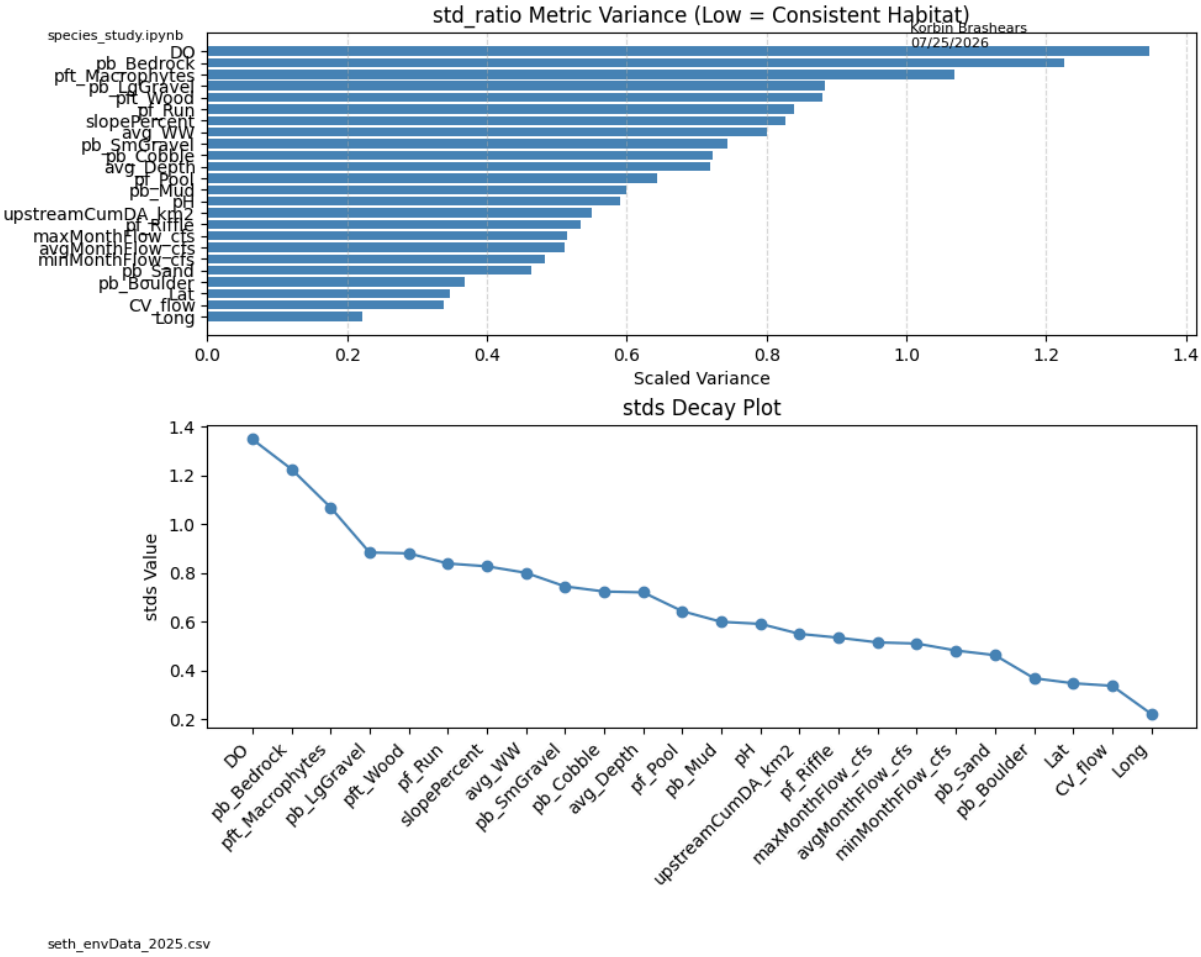
> Comparing the ratio of the Std of each environmental metric with the Std for only the C. Anomalum sites.

In [33]:
```
base_std = np.std(metrics_all, axis=0)
filtered_std = np.std(metrics_filtered, axis=0)

std_m = pd.concat([base_std, filtered_std], axis=1)

std_m['std_ratio'] = std_m[1]/std_m[0]

print(std_m)
```

```
                           0           1  std_ratio
Lat                 0.295146    0.102614   0.347674
Long                0.769675    0.171028   0.222208
DO                  2.895852    3.901282   1.347197
pH                  0.583217    0.344708   0.591045
avg_WW              4.101073    3.281298   0.800107
avg_Depth          12.575821    9.054595   0.720000
pf_Pool            22.446256   14.433755   0.643036
pf_Run             22.086560   18.517072   0.838386
pf_Riffle          15.099368    8.068717   0.534374
pb_Mud             34.850670   20.885940   0.599298
pb_Sand            11.151883    5.162122   0.462892
pb_SmGravel        12.847268    9.564575   0.744483
pb_LgGravel        10.420673    9.204678   0.883309
pb_Cobble           8.150498    5.897550   0.723581
pb_Boulder         10.563794    3.889549   0.368196
pb_Bedrock         17.237409   21.116744   1.225053
pft_Macrophytes    30.959930   33.071891   1.068216
pft_Wood           32.026684   28.182811   0.879979
upstreamCumDA_km2 224.491729  123.482126   0.550052
slopePercent        0.376918    0.311649   0.826835
avgMonthFlow_cfs   68.304742   34.861782   0.510386
CV_flow             0.035365    0.011924   0.337182
maxMonthFlow_cfs  141.784905   73.029069   0.515069
minMonthFlow_cfs   17.607917    8.484682   0.481867
```

In [34]:
```
# 3. Compute variance within anomalum sites
stds = std_m['std_ratio'].sort_values(ascending=False)
#stds = pd.Series(variances, index=std_m.columns).sort_values()

# 4. Plot
plt.figure(figsize=(10, 8))
plt.subplot(2, 1, 1)
plt.barh(stds.index, stds.values, color="steelblue")
plt.xlabel("Scaled Variance")
plt.title(f"std_ratio Metric Variance (Low = Consistent Habitat)")
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.gca().invert_yaxis()

plt.subplot(2, 1, 2)

plt.plot(
```

```
    stds.index,
    stds.values,
    marker='o',
    color="steelblue"
)

plt.xticks(stds.index, rotation=45, ha='right')
plt.ylabel("stds Value")
plt.title("stds Decay Plot")

plt.tight_layout()

lpe(
    author_names="Korbin Brashears",
    data_filenames=filenames['s_ed'],
    fig_filename=f"../figures/niche_variance_std_ratio.png",
    save=False
)
```



seth_envData_2025.csv

Std Results Interpretation

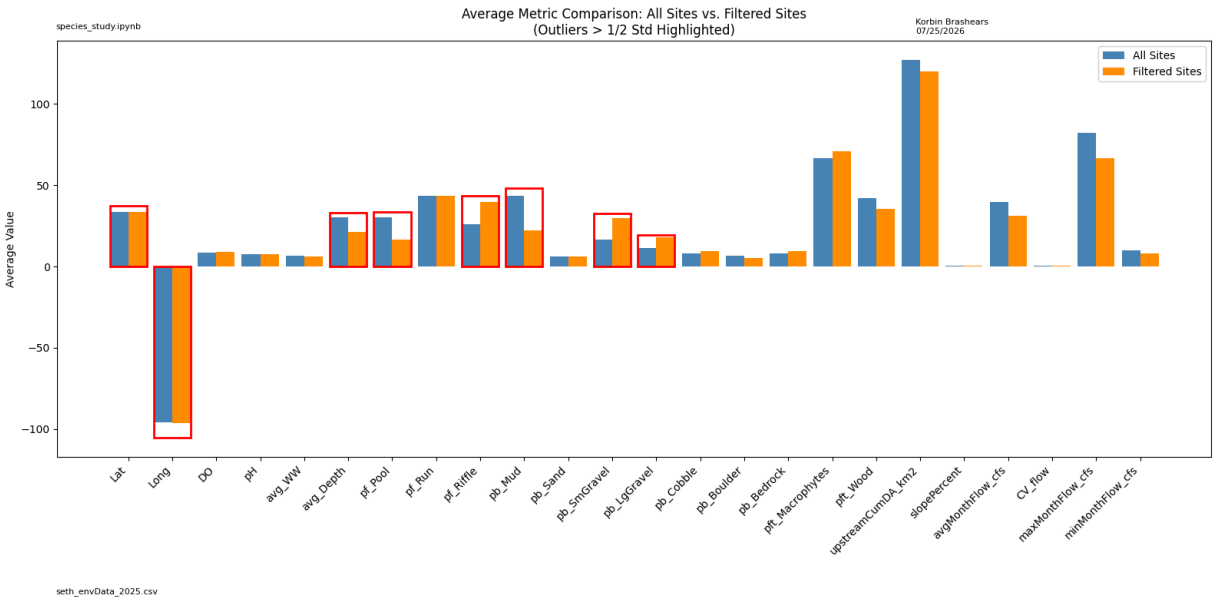
> It seems like these results generally line up with our previous conclusion that flow, channel shape, and bed composition matter the most, as they vary the least within these sites. We see a very steep drop-off in standard deviation

> after DO, Bedrock, and Macrophytes. This is somewhat surprising because we expected Macrophytes and Bedrock to be larger contributors.
>
> It is important to note that this does *not* mean these variables are unimportant, only that they are likely not the primary drivers of variation within the filtered sites.
>
> We can also see that, as expected, CV_flow along with min, average, and upstream flow metrics are largely consistent across these sites. Additionally, pf_riffle, pb_boulder, and Sand appear consistent as well. The remaining variables—average depth, small gravel, pf_run, and others, fall into a middle range of variability but are still much more consistent than DO, Bedrock, and Macrophytes.

> I would like to next plot a bar chart comparing the average of each metric across *all* sites versus the averages across only our filtered sites to better visualize these differences.

In [35]:
```
# Compute means
mean_all = metrics_all.mean()
mean_filtered = metrics_filtered.mean()

# Compute std across ALL sites (used as threshold)
std_all = metrics_all.std()

# Difference between filtered and all
diff = (mean_filtered - mean_all).abs()

# Identify metrics where difference > 1/2 std
outlier_metrics = diff[diff > std_all*0.5].index.tolist()

# Combine into a DataFrame for plotting
mean_df = pd.DataFrame({
    "All Sites": mean_all,
    "Filtered Sites": mean_filtered,
    "Diff": diff,
    "Std_All": std_all
}) # X positions
x = np.arange(len(mean_df))
width = 0.42

plt.figure(figsize=(16, 8))

# Bars
bars_all = plt.bar(x - width/2, mean_df["All Sites"],
                   width=width, label="All Sites", color="steelblue")

bars_filtered = plt.bar(x + width/2, mean_df["Filtered Sites"],
                        width=width, label="Filtered Sites", color="darkorange")
```

```
# Highlight metrics where difference > 1 std
for i, metric in enumerate(mean_df.index):
    if metric in outlier_metrics:
        # Draw a red rectangle around both bars for this metric
        rect = patches.Rectangle(
            (i - width, 0),      # x-position (slightly left of both bars)
            width*2,             # width covering both bars
            max(mean_df.loc[metric, ["All Sites", "Filtered Sites"]]) * 1.1,  # hei
            linewidth=2,
            edgecolor='red',
            facecolor='none'
        )
        plt.gca().add_patch(rect)

# Labels
plt.xticks(x, mean_df.index, rotation=45, ha='right')
plt.ylabel("Average Value")
plt.title("Average Metric Comparison: All Sites vs. Filtered Sites\n(Outliers > 1/2
plt.legend()

plt.tight_layout()

lpe(
    author_names="Korbin Brashears",
    data_filenames=filenames['s_ed'],
    fig_filename="../figures/niche_variance_std_ratio.png",
    save=False
)

plt.show()
```



Interpretation of Average Differences Between All Sites and Filtered Sites

The comparison between the average environmental metrics across *all* sites and those from only the filtered sites containing **N ≥ 10 C. anomalum** reveals a consistent and ecologically meaningful pattern. Several substrate and channel-shape variables shift in ways that align with known habitat preferences of *Campostoma anomalum*, even though many of these differences were not flagged by the 1/2-standard-deviation threshold used in the variance analysis.

Substrate and Channel Characteristics

Across the filtered sites, we observe:

- **Lower average depth**, indicating a preference for shallow water.
- **Substantially lower pool proportion**, nearly half of the full-dataset average.
- **Higher riffle proportion**, roughly one-quarter greater than all sites.
- **Lower mud proportion**, approximately half of the full-dataset average.
- **Higher small-gravel proportion**, nearly double, which is the strongest substrate signal.
- **Moderately higher large-gravel proportion**, consistent with stable riffle substrates.

Together, these shifts outline a clear habitat signature: shallow, fast-moving riffle environments with stable gravel substrates and reduced fine sediment. These conditions are well documented as essential for feeding and spawning in *C. anomalum*.

Flow Characteristics

Flow-related metrics show a more subtle pattern:

- **Upstream flow, average monthly flow, max flow, and min flow** are all slightly lower in filtered sites.
- **CV_flow is identical** between groups.
- **General flow appears lower overall** in occupied sites.

Although initially surprising, this pattern is consistent with riffle ecology. *C. anomalum* does not require high discharge; instead, it relies on **stable riffle conditions** where gravel remains in place and fine sediment does not accumulate. Excessively high flow can scour gravel, while very low flow promotes mud and sand deposition. The filtered sites appear to represent moderate-flow environments that maintain substrate stability rather than extremes of discharge.

Vegetation and Wood

- **Macrophytes are slightly higher** in filtered sites, which may reflect stable substrate conditions in moderate-flow reaches.
- **Wood proportion is slightly lower**, consistent with riffles where woody debris is less likely to accumulate.

These differences are small but fit the broader riffle-versus-pool contrast observed in the substrate metrics.

Why PCA Loadings and Niche Variance Did Not Flag These Metrics

Several of the ecologically meaningful differences-such as lower depth, higher riffle, reduced mud, and increased small gravel-were not flagged by the 1-standard-deviation threshold. This is because PCA and niche variance highlight variables that **explain variance**, not variables that **explain ecological preference**. Metrics like CV_flow and upstream flow were flagged only because they are extremely consistent across filtered sites, not because they are biologically influential. The mean-difference comparison provides a complementary perspective, revealing ecological signals that PCA cannot capture.

Emerging Habitat Blueprint for *C. anomalum* Sites (N ≥ 10)

The filtered sites share a coherent environmental profile:

- Lower average depth
- Lower pooling
- Higher riffle proportion
- Lower mud
- Higher small gravel
- Slightly higher large gravel
- Moderate, stable flow conditions

This combination strongly supports the conclusion that *C. anomalum* occupies shallow, riffle-dominated habitats with stable gravel substrates and minimal fine sediment. The mean-difference analysis reinforces and clarifies the ecological interpretation suggested by the PCA and variance results.

Testing Our Results by Scoring Each Site's Feature–Population Combinations

> To evaluate whether our final habitat hypothesis matches the actual site-level data, we will create a function that scores each site according to our expected "blueprint" features. Each feature will be scaled from **0 to 1**, where **0** corresponds to the minimum value of that feature across *all* sites and **1** corresponds to the maximum value across all sites. These min–max values will come from `metrics_all`, not from `metrics_filtered`, so that the scoring reflects the full environmental range of the dataset.

> For each site, the function will compute a 0–1 score for every blueprint feature (e.g., depth, riffle, mud, small gravel, large gravel, flow metrics). These standardized values will be stored as a row in a new DataFrame. The final output will contain all 46 sites, each with:
>
> - the site number
> - the 0–1 scaled feature scores
> - the *C. anomalum* count for that site
>
> This will allow us to visually inspect whether the feature combinations at each site align with our ecological expectations. If the general patterns appear correct—meaning high scores for riffle and small gravel correspond to higher *C. anomalum* counts, while high mud or deep pooling correspond to lower counts—then we can proceed to the next step.

> Once the scoring system is validated, we will extend it by creating a **weighted "population score" function**. This function will multiply each standardized feature by a weight representing its hypothesized importance (e.g., small gravel weighted more heavily than macrophytes). The resulting weighted score will estimate the likelihood that a site supports high *C. anomalum* abundance. This will serve as a preliminary habitat suitability index that can be refined or validated with additional modeling.

In [36]:
```python
# Precompute min and max for all features across ALL sites
feature_min = metrics_all.min()
feature_max = metrics_all.max()

def scale_site_features(site_row):
    """
    Takes a row from metrics_all or metrics_filtered and returns
    a dict of 0-1 scaled feature values using global min/max.
    """
    scaled = (site_row - feature_min) / (feature_max - feature_min)
    return scaled.round(2)   # round to 2 decimals as requested
```

In [37]:
```python
# Extract the site numbers
site_numbers = s_ed["SiteN"]

# Compute scaled features for each site
scaled_rows = []
for idx, row in metrics_all.iterrows():
    scaled = scale_site_features(row)
    scaled["SiteN"] = site_numbers.loc[idx]
    scaled["C_anomalum_count"] = s_fcd['Campostoma_anomalum'].loc[idx]
    scaled_rows.append(scaled)

# Final DataFrame
```

```
scaled_df = pd.DataFrame(scaled_rows)

# Reorder columns: SiteN, C_anomalum_count, then features
cols = ["SiteN", "C_anomalum_count"] + [c for c in scaled_df.columns if c not in ["
scaled_df = scaled_df[cols]

scaled_df.head()
```

Out[37]:

	SiteN	C_anomalum_count	Lat	Long	DO	pH	avg_WW	avg_Depth	pf_Pool	pf_Ru
0	1.0	1.0	0.23	0.32	0.45	0.26	0.17	0.34	0.00	1.0
1	2.0	0.0	0.21	0.33	0.00	0.33	0.02	0.40	0.60	0.2
2	3.0	0.0	0.24	0.29	0.22	0.43	0.45	0.75	0.67	0.4
3	4.0	36.0	0.07	0.23	0.24	0.51	0.41	0.25	0.00	0.8
4	5.0	0.0	0.30	0.35	0.18	0.39	1.00	0.63	1.00	0.0

5 rows × 26 columns

In [38]:

```
# Example weights based on your blueprint interpretation
weights = {
    "avg_Depth": -1.0,          # shallower sites preferred
    "pf_Pool": -1.5,            # less pooling strongly preferred
    "pf_Riffle": 2.0,           # riffle habitat highly preferred
    "pb_Mud": -2.0,             # mud strongly avoided
    "pb_SmGravel": 2.5,         # small gravel extremely important (spawning substr
    "pb_LgGravel": 1.0,         # large gravel moderately helpful
    "avgMonthFlow_cfs": 0.5,    # moderate flow beneficial
    "CV_flow": 0.0,             # flow consistency not predictive
    "pf_Run": 0.5,              # run habitat moderately helpful
    "pb_Boulder": 0.5,          # stable substrate helpful
    "pb_Sand": -1.0,            # sand mildly avoided
    "pft_Macrophytes": 0.2,     # slight positive association
    "pft_Wood": -0.2,           # wood slightly avoided (accumulates in pools)
    "maxMonthFlow_cfs": -0.3,   # extreme high flows harmful (scour)
    "minMonthFlow_cfs": -0.3,   # extreme low flows harmful (sediment deposition)
    "upstreamCumDA_km2": 0.1,   # slight positive drainage area effect
    "slopePercent": 0.1         # slight positive slope effect (riffle formation)
}

def compute_population_score(row, weights):
    score = 0
    for feature, weight in weights.items():
        if feature in row.index:
            score += row[feature] * weight
    return score   # keep full precision

# Apply raw score
scaled_df["population_score_raw"] = scaled_df.apply(
    lambda r: compute_population_score(r, weights), axis=1
)
```

```
# Normalize raw score to 0-1 probability-like score
min_score = scaled_df["population_score_raw"].min()
max_score = scaled_df["population_score_raw"].max()

scaled_df["population_score"] = (
    (scaled_df["population_score_raw"] - min_score) /
    (max_score - min_score)
).round(3)

scaled_df.head()
```

Out[38]:

	SiteN	C_anomalum_count	Lat	Long	DO	pH	avg_WW	avg_Depth	pf_Pool	pf_Ru
0	1.0	1.0	0.23	0.32	0.45	0.26	0.17	0.34	0.00	1.0
1	2.0	0.0	0.21	0.33	0.00	0.33	0.02	0.40	0.60	0.2
2	3.0	0.0	0.24	0.29	0.22	0.43	0.45	0.75	0.67	0.4
3	4.0	36.0	0.07	0.23	0.24	0.51	0.41	0.25	0.00	0.8
4	5.0	0.0	0.30	0.35	0.18	0.39	1.00	0.63	1.00	0.0

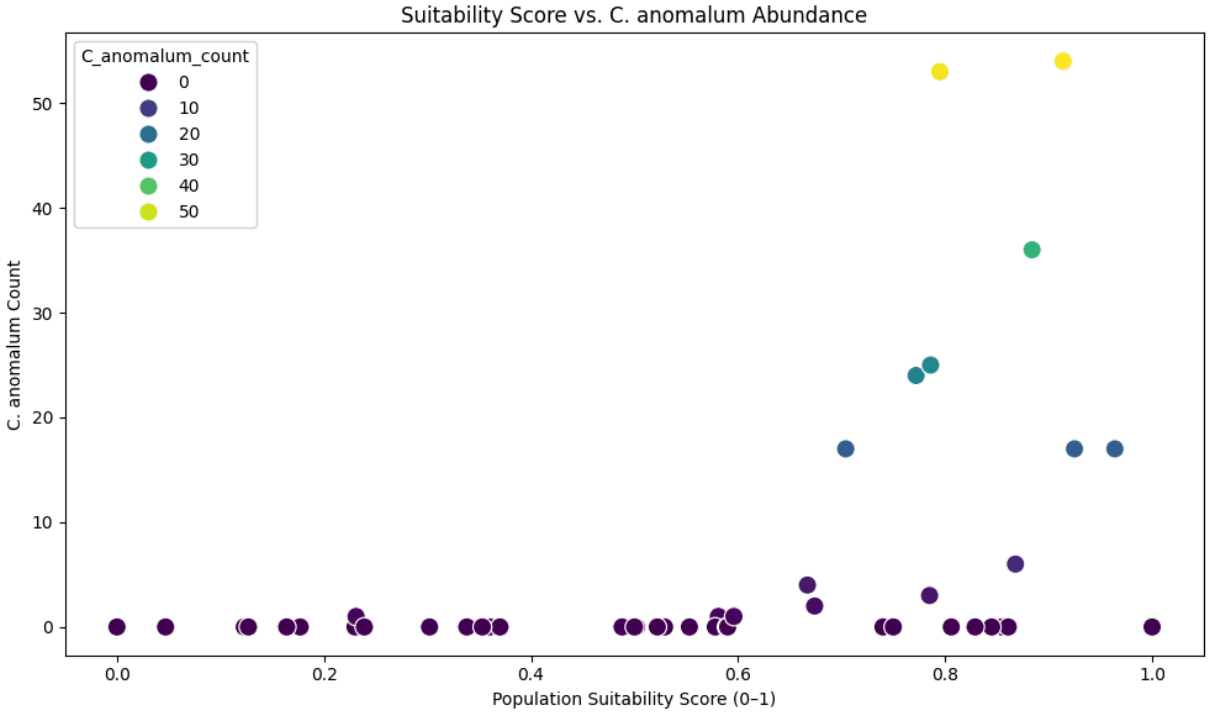
5 rows × 28 columns

In [39]:
```
scaled_df.to_csv('../data/species_study/scaled_1to0_rating.csv')
```

In [40]:
```
print(scaled_df[["population_score", "C_anomalum_count"]].corr())
```

```
                  population_score  C_anomalum_count
population_score          1.000000          0.441385
C_anomalum_count          0.441385          1.000000
```

In [41]:
```
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=scaled_df,
    x="population_score",
    y="C_anomalum_count",
    hue="C_anomalum_count",
    palette="viridis",
    s=120
)

plt.title("Suitability Score vs. C. anomalum Abundance")
plt.xlabel("Population Suitability Score (0-1)")
plt.ylabel("C. anomalum Count")
plt.tight_layout()
plt.show()
```



> Using our weighted scores to create a binary classify for N>=10 C. Anomalum.

In [42]:
```
# Creating a binary column for C. anomalum presence based on a threshold of 10 indi

scaled_df["C_anomalum_binary"] = (scaled_df["C_anomalum_count"] >= 10).astype(int)
```

In [43]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

X = scaled_df[["population_score"]]   # single predictor
y = scaled_df["C_anomalum_binary"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_st

clf = LogisticRegression()
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
```

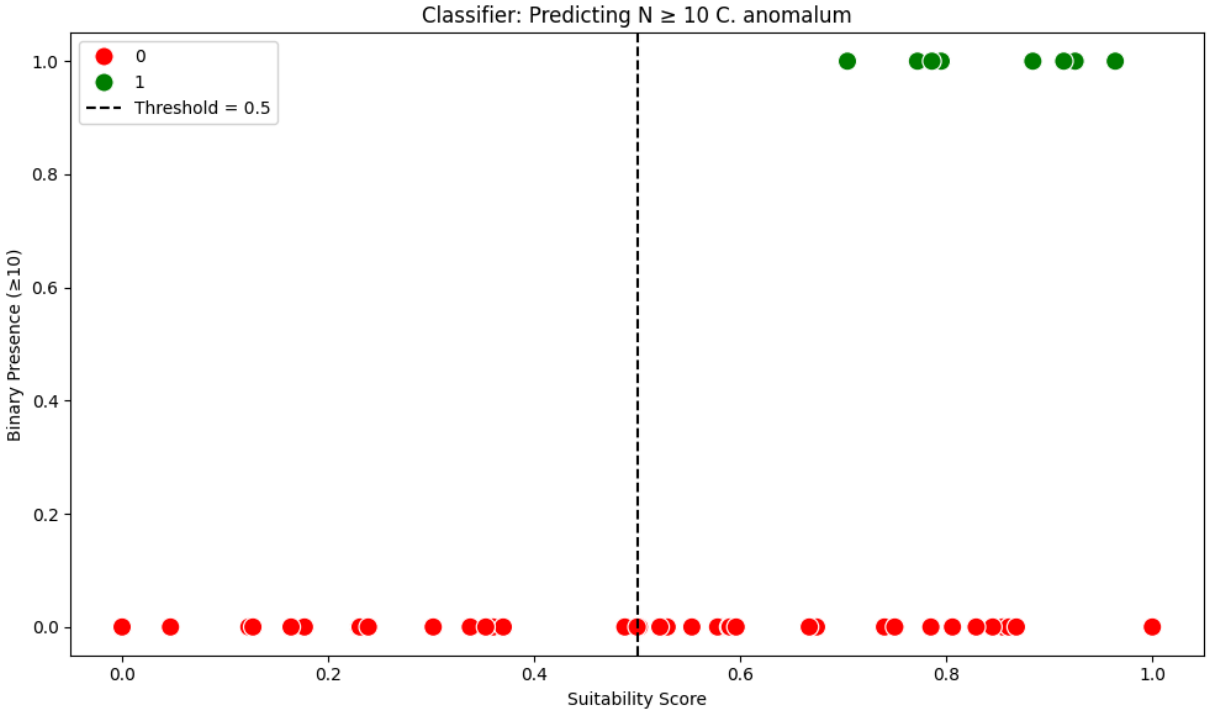
In [44]:
```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.75
[[9 0]
 [3 0]]
              precision    recall  f1-score   support

           0       0.75      1.00      0.86         9
           1       0.00      0.00      0.00         3

    accuracy                           0.75        12
   macro avg       0.38      0.50      0.43        12
weighted avg       0.56      0.75      0.64        12
```

```
c:\Users\korbi\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\korbi\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\korbi\miniconda3\Lib\site-packages\sklearn\metrics\_classification.py:1833:
UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in labels with
no predicted samples. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In [45]:

```
plt.figure(figsize=(10,6))
sns.scatterplot(
    data=scaled_df,
    x="population_score",
    y="C_anomalum_binary",
    hue="C_anomalum_binary",
    palette=["red","green"],
    s=120
)

plt.axvline(0.5, color="black", linestyle="--", label="Threshold = 0.5")
plt.title("Classifier: Predicting N ≥ 10 C. anomalum")
plt.xlabel("Suitability Score")
plt.ylabel("Binary Presence (≥10)")
plt.legend()
plt.tight_layout()
plt.show()
```



> From these plots, we can clearly see that our current suitability score can do a good job of showing whether or not a site is not inhabitable by Campsotma Anomalum; however, it also shows that, while a site may be habitable, that does not necessarily mean that it will be inhabited.

> From this analysis I would like to propose to myself that we should include a separate metric perhaps from trying to predict whether there will be fish but rather could there be. Essentially I would like to try to identify if a given site is not inhabitable, and what make it so.

> Creating a weight tuning function to find the highest correlation between Suitability Score and population.

In [46]:
```
def evolve_weights(
    scaled_df,
    FEATURE_NAMES,
    pop_size=50,
    generations=200,
    mutation_rate=0.15,
    weight_range=(-3, 3),
    seed=42
):
    np.random.seed(seed)

    # Extract abundance vector
    y = scaled_df["C_anomalum_count"].values
```

```
    # --- Initialize population of random weight sets ---
    population = [
        {feat: np.random.uniform(*weight_range) for feat in FEATURE_NAMES}
        for _ in range(pop_size)
    ]

    def fitness(weights):
        raw = scaled_df.apply(
            lambda r: sum(r[f] * weights[f] for f in FEATURE_NAMES),
            axis=1
        )
        norm = (raw - raw.min()) / (raw.max() - raw.min())
        return np.corrcoef(norm, y)[0, 1]

    best_corr = -999
    c_anomalum_weights = None

    for gen in range(generations):

        # --- Evaluate fitness of each weight set ---
        scored = [(fitness(w), w) for w in population]
        scored.sort(key=lambda x: x[0], reverse=True)

        # Track best
        if scored[0][0] > best_corr:
            best_corr = scored[0][0]
            c_anomalum_weights = scored[0][1]

        # --- Select top 20% as parents ---
        parents = [w for _, w in scored[:pop_size // 5]]

        # --- Create new population via crossover ---
        new_population = []

        while len(new_population) < pop_size:
            p1, p2 = np.random.choice(parents, 2, replace=True)
            child = {}

            for feat in FEATURE_NAMES:
                # crossover: mix parent weights
                if np.random.rand() < 0.5:
                    child[feat] = p1[feat]
                else:
                    child[feat] = p2[feat]

                # mutation
                if np.random.rand() < mutation_rate:
                    child[feat] += np.random.uniform(-1, 1)

            new_population.append(child)

        population = new_population

    return best_corr, c_anomalum_weights
```

In [47]:
```
# best_corr, c_anomalum_weights = evolve_weights(
#     scaled_df,
#     FEATURE_NAMES,
#     generations=400
#     )

# print("Best Correlation:", best_corr)
# print("Best Weights:", c_anomalum_weights)
```

> Adding meaningful weight bounds for our alg to follow.

In [48]:
```
weight_bounds = {
    "avg_Depth": (-3, -0.1),        # must be negative
    "pf_Pool": (-4, -0.5),          # must be strongly negative
    "pf_Riffle": (0.5, 4),          # must be positive
    "pb_Mud": (-4, -0.5),           # must be strongly negative
    "pb_SmGravel": (1, 5),          # must be strongly positive
    "pb_LgGravel": (0, 2),          # mild positive
    "avgMonthFlow_cfs": (-1, 1),    # moderate effect
    "CV_flow": (-1, 1),             # weak effect
    "pf_Run": (0, 2),               # mild positive
    "pb_Boulder": (0, 2),           # mild positive
    "pb_Sand": (-2, 0),             # negative
    "pft_Macrophytes": (-1, 1),     # weak effect
    "pft_Wood": (-1, 1),            # weak effect
    "maxMonthFlow_cfs": (-2, 0),    # negative
    "minMonthFlow_cfs": (-1, 1),    # weak effect
    "upstreamCumDA_km2": (-1, 1),   # weak effect
    "slopePercent": (0, 2)          # positive
}
```

In [49]:
```
def evolve_weights_constrained(
    scaled_df,
    FEATURE_NAMES,
    weight_bounds,
    pop_size=200,
    generations=200,
    mutation_rate=0.15,
    seed=42
):
    np.random.seed(seed)
    y = scaled_df["C_anomalum_count"].values

    # --- Initialize population ---
    def random_weights():
        return {
            feat: np.random.uniform(*weight_bounds[feat])
            for feat in FEATURE_NAMES
        }

    population = [random_weights() for _ in range(pop_size)]

    def fitness(weights):
```

```
        # Compute raw score
        raw = scaled_df.apply(
            lambda r: sum(r[f] * weights[f] for f in FEATURE_NAMES),
            axis=1
        )
        norm = (raw - raw.min()) / (raw.max() - raw.min())
        corr = np.corrcoef(norm, y)[0, 1]

        # Penalty for violating ecological expectations
        penalty = 0

        # Small gravel MUST be high
        if weights["pb_SmGravel"] < 1.5:
            penalty -= 0.2

        # Riffle MUST be positive
        if weights["pf_Riffle"] < 0.5:
            penalty -= 0.2

        # Mud MUST be negative
        if weights["pb_Mud"] > -0.5:
            penalty -= 0.2

        # Pool MUST be strongly negative
        if weights["pf_Pool"] > -0.5:
            penalty -= 0.2

        # Prevent extreme magnitudes
        for feat in FEATURE_NAMES:
            low, high = weight_bounds[feat]
            if not (low <= weights[feat] <= high):
                penalty -= 0.5

        return corr + penalty

    best_corr = -999
    c_anomalum_weights = None

    for gen in range(generations):

        scored = [(fitness(w), w) for w in population]
        scored.sort(key=lambda x: x[0], reverse=True)

        if scored[0][0] > best_corr:
            best_corr = scored[0][0]
            c_anomalum_weights = scored[0][1]

        parents = [w for _, w in scored[:pop_size // 5]]

        new_population = []
        while len(new_population) < pop_size:
            p1, p2 = np.random.choice(parents, 2, replace=True)
            child = {}

            for feat in FEATURE_NAMES:
                child[feat] = p1[feat] if np.random.rand() < 0.5 else p2[feat]
```

```
                # mutation
                if np.random.rand() < mutation_rate:
                    low, high = weight_bounds[feat]
                    child[feat] = np.random.uniform(low, high)

            new_population.append(child)

        population = new_population

    return best_corr, c_anomalum_weights
```

In [50]:
```
# best_corr, c_anomalum_weights = evolve_weights_constrained(
#     scaled_df,
#     FEATURE_NAMES,
#     weight_bounds
# )

# print("Best Corr:", best_corr)
# print("Best Weights:", c_anomalum_weights)
```

In [51]:
```
# y = scaled_df['C_anomalum_count'].values

# # vectorized raw score
# w = np.array([c_anomalum_weights[f] for f in FEATURE_NAMES])
# X = scaled_df[FEATURE_NAMES].values
# raw = np.sum(X * w, axis=1)

# # normalize

# corr = np.corrcoef(raw, y)[0, 1]

# print("Final correlation:", corr)
```

> Creating a better algorithm.
>
> This one will utilize a path-finding local maxima with random stepping to try to find the best correlation point in the 17dim space.

In [52]:
```
from joblib import Parallel, delayed

# ---------- GLOBALS ----------
GLOBAL_DF = None
GLOBAL_FEATURES = None
GLOBAL_BOUNDS = None

# ---------- RAW CORRELATION (no penalties) ----------
def raw_corr(weights, binary=False):
    df = GLOBAL_DF
    FEATURE_NAMES = GLOBAL_FEATURES
```

```
    if binary:
        y = df['C_anomalum_binary'].values
    else:
        y = df['C_anomalum_count'].values

    w = np.array([weights[f] for f in FEATURE_NAMES])
    X = df[FEATURE_NAMES].values

    raw = np.sum(X * w, axis=1)
    rmin, rmax = raw.min(), raw.max()
    if rmax == rmin:
        return None

    norm = (raw - rmin) / (rmax - rmin)
    return np.corrcoef(norm, y)[0, 1]

# ---------- PENALTIES (applied AFTER climbing) ----------
def apply_penalties(weights):
    penalty = 0
    if weights["pb_SmGravel"] < 1.5:
        penalty -= 0.2
    if weights["pf_Riffle"] < 0.5:
        penalty -= 0.2
    if weights["pb_Mud"] > -0.5:
        penalty -= 0.2
    if weights["pf_Pool"] > -0.5:
        penalty -= 0.2

    for feat in GLOBAL_FEATURES:
        low, high = GLOBAL_BOUNDS[feat]
        if not (low <= weights[feat] <= high):
            penalty -= 0.5

    return penalty

# ---------- RANDOM START ----------
def random_weights():
    return {
        feat: np.random.uniform(*GLOBAL_BOUNDS[feat])
        for feat in GLOBAL_FEATURES
    }

# ---------- OPTIMIZED HILL-CLIMB WORKER ----------
def hill_climb_worker(seed):
    np.random.seed(seed)

    weights = random_weights()
    best_corr = raw_corr(weights, binary=GLOBAL_BINARY)

    if best_corr is None:
        return (None, None)

    # adaptive step schedule
    step_sizes = [1.0, 0.5, 0.25, 0.1, 0.05, 0.01, 0.005, 0.001]

    for step in step_sizes:
```

```
        improved = True

        while improved:
            improved = False

            # diagonal stepping: adjust multiple features at once
            for feat in GLOBAL_FEATURES:
                low, high = GLOBAL_BOUNDS[feat]

                # +step
                w_plus = weights.copy()
                w_plus[feat] = min(w_plus[feat] + step, high)
                c_plus = raw_corr(w_plus)

                # -step
                w_minus = weights.copy()
                w_minus[feat] = max(w_minus[feat] - step, low)
                c_minus = raw_corr(w_minus)

                # choose best direction
                if c_plus is not None and c_plus > best_corr:
                    weights = w_plus
                    best_corr = c_plus
                    improved = True
                    continue

                if c_minus is not None and c_minus > best_corr:
                    weights = w_minus
                    best_corr = c_minus
                    improved = True
                    continue

    # apply penalties AFTER climbing
    final_score = best_corr + apply_penalties(weights)
    return (weights, final_score)

# ---------- PARALLEL DRIVER ----------
def parallel_hill_climb(
    scaled_df,
    FEATURE_NAMES,
    weight_bounds,
    n_parallel=8,
    n_restarts_per_worker=2,
    binary=False
):
    global GLOBAL_DF, GLOBAL_FEATURES, GLOBAL_BOUNDS, GLOBAL_BINARY
    GLOBAL_DF = scaled_df
    GLOBAL_FEATURES = FEATURE_NAMES
    GLOBAL_BOUNDS = weight_bounds
    GLOBAL_BINARY = binary

    seeds = list(range(n_parallel * n_restarts_per_worker))

    results = Parallel(n_jobs=n_parallel, backend="loky")(
        delayed(hill_climb_worker)(seed) for seed in seeds
    )
```

```
    c_anomalum_weights = None
    best_score = -999

    for w, s in results:
        if s is None:
            continue
        if s > best_score:
            best_score = s
            c_anomalum_weights = w

    return best_score, c_anomalum_weights
```

> Trying with raw count.

In [53]:
```
# best_corr, c_anomalum_weights = parallel_hill_climb(
#     scaled_df,
#     FEATURE_NAMES,
#     weight_bounds,
#     n_parallel=8,
#     n_restarts_per_worker=20,
# )
# print("Best Corr (with penalties):", best_corr)
# print("Best Weights:", c_anomalum_weights)
```

In [54]:
```
# y = scaled_df['C_anomalum_count'].values

# # vectorized raw score
# w = np.array([c_anomalum_weights[f] for f in FEATURE_NAMES])
# X = scaled_df[FEATURE_NAMES].values
# raw = np.sum(X * w, axis=1)

# # normalize

# corr = np.corrcoef(raw, y)[0, 1]

# print("Final correlation:", corr)
```

In [55]:
```
# y = scaled_df['C_anomalum_count'].values

# # vectorized raw score
# w = np.array([c_anomalum_weights[f] for f in FEATURE_NAMES])
# X = scaled_df[FEATURE_NAMES].values
# raw = np.sum(X * w, axis=1)

# # normalize

# corr = np.corrcoef(raw, y)[0, 1]

# print("Final correlation:", corr)
```

In [56]:
```
# y = scaled_df['C_anomalum_binary'].values
```

```
# # vectorized raw score
# w = np.array([c_anomalum_weights[f] for f in FEATURE_NAMES])
# X = scaled_df[FEATURE_NAMES].values
# raw = np.sum(X * w, axis=1)

# # normalize

# corr = np.corrcoef(raw, y)[0, 1]

# print("Final correlation:", corr)
```

> Trying with binary presence.

In [57]:
```
# best_corr, c_anomalum_weights = parallel_hill_climb(
#     scaled_df,
#     FEATURE_NAMES,
#     weight_bounds,
#     n_parallel=8,
#     n_restarts_per_worker=20,
#     binary=True
# )
# print("Best Corr (with penalties):", best_corr)
# print("Best Weights:", c_anomalum_weights)
```

In [58]:
```
# y = scaled_df['C_anomalum_binary'].values

# # vectorized raw score
# w = np.array([c_anomalum_weights[f] for f in FEATURE_NAMES])
# X = scaled_df[FEATURE_NAMES].values
# raw = np.sum(X * w, axis=1)

# # normalize

# corr = np.corrcoef(raw, y)[0, 1]

# print("Final correlation:", corr)
```

Moving to a GA (genetic algorithm) for optimization of weights.

> To identify which environmental metrics best predict species presence or abundance, we used a genetic algorithm (GA) to evolve an optimal set of feature weights.

> The GA evaluates thousands of candidate weight vectors and selects those that maximize the correlation between a weighted combination of environmental metrics and the target species response.

> This approach is well suited for ecological data because it does not assume linearity, normality, or independence among predictors. The resulting weight

> vector provides a direct measure of each metric's predictive contribution.

How does GA work

- **Initialize a population**
  Create an initial set of candidate solutions (e.g., vectors of feature weights). Each individual represents one possible answer.
- **Define a fitness function**
  Evaluate every individual by scoring how well it performs on the task - such as how strongly its weighted feature combination correlates with the target species metric.
- **Selection**
  Choose the best-performing individuals based on their fitness scores. These become the "parents" for the next generation.
- **Evolution (Crossover)**
  Combine pairs of selected parents to produce new "children." Each child inherits information from both parents, forming new candidate solutions.
- **Mutation**
  Randomly perturb some of the children's values. Mutation helps prevent the algorithm from getting stuck in local optima and encourages exploration of new solutions.
- **Repetition**
  Repeat evaluation, selection, crossover, and mutation for many generations. Over time, the population evolves toward an optimal or near-optimal solution.

In [61]:
```
# Moving to a GA algorithm for optimization of weights

# Importing DEAP library for genetic algorithm implementation
# The DEAP (Distributed Evolutionary Algorithms in Python) library is a framework f
# It provides tools for creating genetic algorithms, genetic programming, and other
# Unlike traditional optimization methods, DEAP allows for the evolution of solutio

# DEAP also utilizes Python's object-oriented programming features, allowing users

# DEAP makes us use the concept of "creator" to define the structure of individuals
# It also provides a "toolbox" for registering functions that will be used in the e

from deap import base, creator, tools, algorithms

# Define the fitness function and individual representation

creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", list, fitness=creator.FitnessMax)

toolbox = base.Toolbox()
```

In [62]:
```
from deap import base, creator, tools, algorithms
from scipy.stats import spearmanr

def evaluate_weights(weights, X, y):
    weighted_sum = np.dot(X, weights)
    corr, _ = spearmanr(weighted_sum, y)
    return (abs(corr),)

def run_ga_weight_optimizer(X, y, n_generations=60, pop_size=120):
    n_features = X.shape[1]

    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    creator.create("Individual", list, fitness=creator.FitnessMax)

    toolbox = base.Toolbox()
    toolbox.register("attr_float", np.random.uniform, -1, 1)
    toolbox.register("individual", tools.initRepeat, creator.Individual,
                     toolbox.attr_float, n_features)
    toolbox.register("population", tools.initRepeat, list, toolbox.individual)

    toolbox.register("evaluate", evaluate_weights, X=X, y=y)
    toolbox.register("mate", tools.cxBlend, alpha=0.5)
    toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.3, indpb=0.2)
    toolbox.register("select", tools.selTournament, tournsize=3)

    pop = toolbox.population(pop_size)
    hof = tools.HallOfFame(1)

    # Stats to track
    stats = tools.Statistics(lambda ind: ind.fitness.values[0])
    stats.register("avg", np.mean)
    stats.register("max", np.max)
    stats.register("min", np.min)
    stats.register("std", np.std)

    # Run GA with logging
    pop, logbook = algorithms.eaSimple(
        pop, toolbox,
        cxpb=0.5, mutpb=0.3,
        ngen=n_generations,
        stats=stats,
        halloffame=hof,
        verbose=True
    )

    return np.array(hof[0]), logbook
```

In [63]:
```
def plot_ga_log(logbook, title="GA Evolution"):
    gens = logbook.select("gen")
    avg = logbook.select("avg")
    maxv = logbook.select("max")
    minv = logbook.select("min")

    plt.figure(figsize=(10, 6))
    plt.plot(gens, avg, label="Average Fitness", color="blue")
```

```
    plt.plot(gens, maxv, label="Max Fitness", color="green")
    plt.plot(gens, minv, label="Min Fitness", color="red")
    plt.xlabel("Generation")
    plt.ylabel("Fitness (Correlation)")
    plt.title(title)
    plt.grid(True)
    plt.legend()
    plt.show()
```

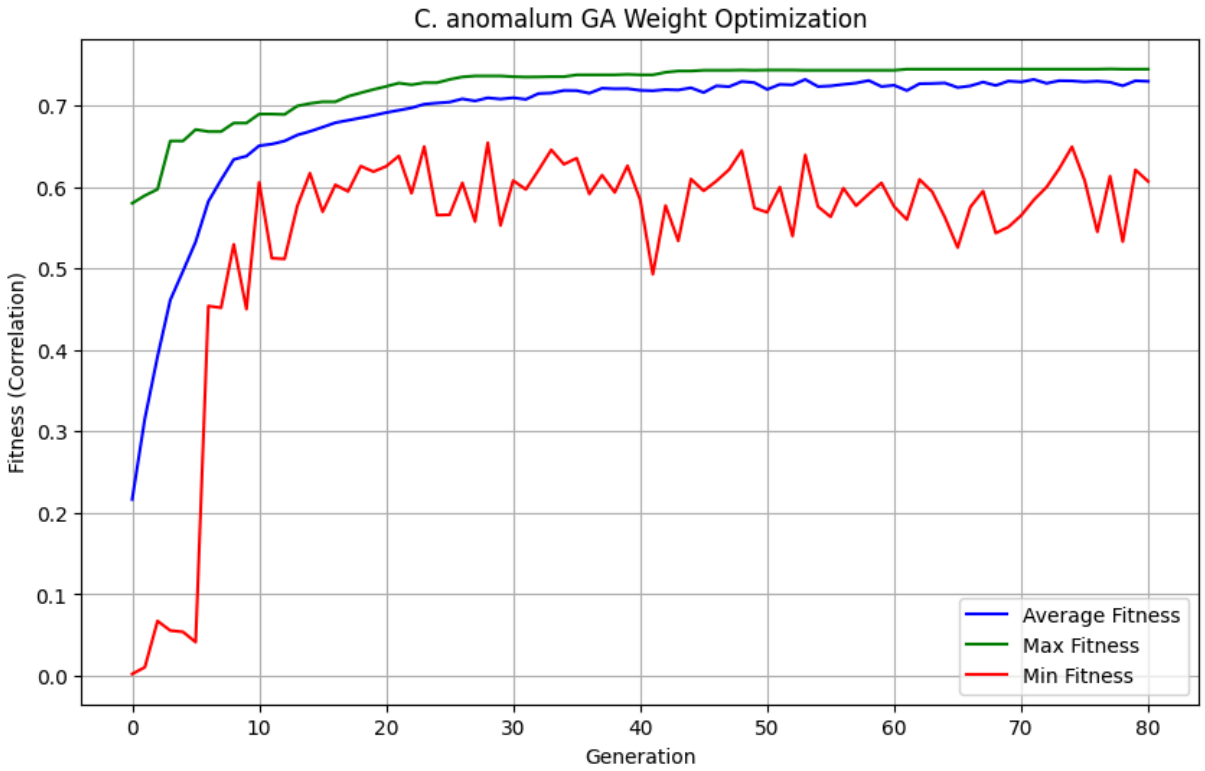
In [64]:
```
c_anomalum_weights, logbook = run_ga_weight_optimizer(
    X=scaled_df[FEATURE_NAMES].values,
    y=scaled_df['C_anomalum_count'].values,
    n_generations=80,
    pop_size=100
)
```

```
c:\Users\korbi\miniconda3\Lib\site-packages\deap\creator.py:185: RuntimeWarning: A c
lass named 'FitnessMax' has already been created and it will be overwritten. Conside
r deleting previous creation of that class or rename it.
  warnings.warn("A class named '{0}' has already been created and it "
c:\Users\korbi\miniconda3\Lib\site-packages\deap\creator.py:185: RuntimeWarning: A c
lass named 'Individual' has already been created and it will be overwritten. Conside
r deleting previous creation of that class or rename it.
  warnings.warn("A class named '{0}' has already been created and it "
```

gen	nevals	avg	max	min	std
0	100	0.216509	0.579883	0.00229554	0.153762
1	52	0.31572	0.589361	0.0105891	0.147543
2	66	0.391966	0.597063	0.0673112	0.121864
3	53	0.461128	0.656302	0.0556113	0.0884151
4	69	0.496545	0.656302	0.0539822	0.0824488
5	67	0.532663	0.670372	0.0413938	0.0961197
6	60	0.582149	0.667854	0.45385	0.0467656
7	66	0.608635	0.667854	0.451629	0.0439913
8	59	0.633674	0.678369	0.529529	0.0236472
9	71	0.637725	0.678369	0.45	0.0277837
10	68	0.650503	0.689328	0.605504	0.0158519
11	59	0.652505	0.689328	0.512498	0.0219597
12	58	0.656292	0.688884	0.511609	0.0240249
13	71	0.663652	0.699251	0.576477	0.0202269
14	68	0.668069	0.702361	0.61676	0.018471
15	76	0.673298	0.704435	0.569516	0.0198985
16	77	0.678736	0.704435	0.602542	0.0197428
17	68	0.681682	0.711099	0.594397	0.0207339
18	71	0.684646	0.715394	0.625498	0.017653
19	59	0.687717	0.719541	0.618685	0.0184838
20	68	0.691157	0.723243	0.625201	0.0191382
21	67	0.694029	0.72739	0.637938	0.0173906
22	69	0.696963	0.725317	0.592175	0.0195154
23	70	0.701459	0.727834	0.64949	0.0165493
24	67	0.702881	0.727834	0.565369	0.0241151
25	68	0.704047	0.731833	0.565814	0.0275274
26	69	0.708008	0.734943	0.604912	0.0223431
27	73	0.705468	0.736128	0.55752	0.0298953
28	79	0.709245	0.736128	0.654081	0.0184385
29	69	0.707777	0.736128	0.552633	0.0295525
30	70	0.709335	0.735239	0.607726	0.0231002
31	71	0.707416	0.734795	0.596914	0.0274241
32	70	0.714544	0.734943	0.62061	0.0210335
33	66	0.715173	0.735239	0.645491	0.0203663
34	52	0.718314	0.735239	0.627719	0.0218899
35	65	0.718123	0.737461	0.635272	0.0198365
36	64	0.714938	0.737461	0.591287	0.0310123
37	56	0.721013	0.737461	0.61439	0.0217233
38	71	0.720388	0.737461	0.593212	0.0223965
39	61	0.720517	0.738053	0.625942	0.0198479
40	68	0.718506	0.737461	0.584178	0.0238885
41	54	0.717989	0.737461	0.492801	0.031926
42	65	0.719399	0.740571	0.577069	0.0286423
43	71	0.718899	0.7422	0.533824	0.0317473
44	69	0.721567	0.7422	0.609651	0.0237594
45	65	0.715769	0.743089	0.595137	0.0341111
46	57	0.723991	0.743089	0.607133	0.0283107
47	67	0.722981	0.743089	0.621203	0.0280766
48	61	0.729332	0.743385	0.644602	0.0230677
49	67	0.727894	0.743089	0.574107	0.0303343
50	76	0.719567	0.743385	0.568776	0.0369959
51	64	0.725764	0.743385	0.599728	0.0307276
52	66	0.725176	0.743385	0.5396	0.0328627
53	66	0.731846	0.743089	0.639271	0.0222225
54	73	0.723122	0.743089	0.575884	0.0339287

55	71	0.723884	0.743089	0.563148	0.0339195
56	77	0.725789	0.743089	0.598395	0.0280446
57	70	0.72739	0.743089	0.576921	0.0293028
58	58	0.730528	0.743089	0.591287	0.0295841
59	70	0.723148	0.743089	0.604912	0.0336555
60	67	0.724661	0.743089	0.575736	0.033035
61	71	0.718195	0.744421	0.55989	0.0410602
62	67	0.726638	0.744421	0.609059	0.0273894
63	60	0.726903	0.744421	0.593952	0.0300436
64	66	0.72748	0.744421	0.562407	0.0316627
65	76	0.721947	0.744421	0.525531	0.0439667
66	71	0.723914	0.744421	0.575292	0.0332489
67	59	0.728603	0.744421	0.594841	0.0303836
68	77	0.724469	0.744421	0.543302	0.0373742
69	59	0.729826	0.744421	0.550856	0.0314751
70	66	0.728741	0.744421	0.564925	0.0327938
71	65	0.731805	0.744421	0.58403	0.0290939
72	80	0.727052	0.744421	0.599728	0.0310568
73	65	0.730318	0.744421	0.622536	0.0271431
74	64	0.730031	0.744421	0.649194	0.0270852
75	74	0.728898	0.744421	0.607874	0.0281984
76	73	0.729726	0.744421	0.544783	0.0320306
77	66	0.728441	0.744718	0.613205	0.0285857
78	77	0.72405	0.744421	0.532787	0.0350065
79	64	0.730222	0.744421	0.620907	0.0291151
80	73	0.729527	0.744421	0.606689	0.0285652

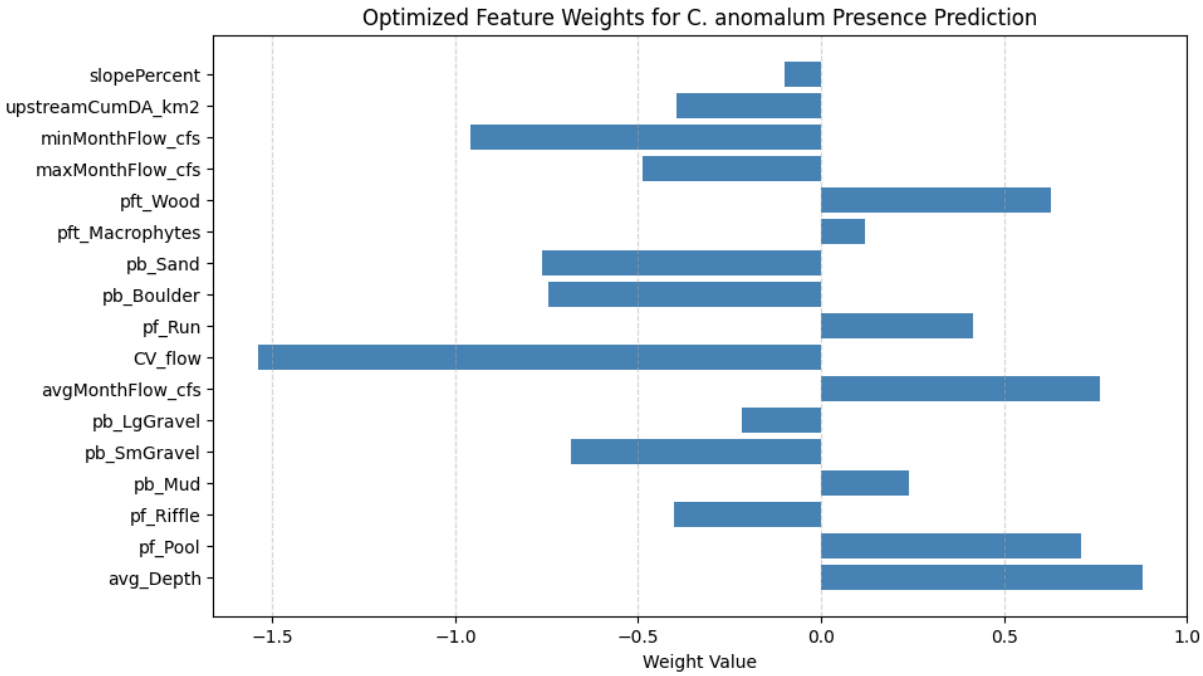
In [65]:
```
plot_ga_log(logbook, title="C. anomalum GA Weight Optimization")
```



In [66]:
```
for name, w in zip(FEATURE_NAMES, c_anomalum_weights):
    print(f"{name}: {w:.4f}")
```

```
avg_Depth: 0.8798
pf_Pool: 0.7094
pf_Riffle: -0.4020
pb_Mud: 0.2411
pb_SmGravel: -0.6847
pb_LgGravel: -0.2149
avgMonthFlow_cfs: 0.7610
CV_flow: -1.5399
pf_Run: 0.4145
pb_Boulder: -0.7460
pb_Sand: -0.7633
pft_Macrophytes: 0.1183
pft_Wood: 0.6278
maxMonthFlow_cfs: -0.4878
minMonthFlow_cfs: -0.9593
upstreamCumDA_km2: -0.3937
slopePercent: -0.0988
```

In [67]:
```
plt.figure(figsize=(10,6))
plt.barh(FEATURE_NAMES, c_anomalum_weights, color="steelblue")
plt.xlabel("Weight Value")
plt.title("Optimized Feature Weights for C. anomalum Presence Prediction")
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.show()
```



Genetic Algorithm for Optimizing Feature Weights

> We use a Genetic Algorithm (GA) to evolve a vector of feature weights that maximizes the correlation between a weighted combination of environmental metrics and the binary presence of *Campostoma anomalum*. Each individual in the GA population represents one possible set of weights. The fitness function applies these weights to the feature matrix and computes the Spearman correlation with the presence vector. The GA then evolves the weights through

> selection, crossover, and mutation to maximize this correlation. The final result is an optimized set of feature weights that indicate which environmental metrics best predict species presence.

In [68]:
```
def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()

def evaluate_weights(weights_raw, X, y):
    """
    Fitness:
    - Convert raw weights to positive, normalized weights via softmax
    - Apply normalized weights to features
    - Compute Spearman correlation with presence
    - Return absolute correlation (maximize)
    """
    weights = softmax(np.array(weights_raw))  # normalized, sum to 1
    weighted_sum = np.dot(X, weights)
    corr, _ = spearmanr(weighted_sum, y)
    return (abs(corr),)
```

In [69]:
```
def run_ga_weight_optimizer(X, y, n_generations=60, pop_size=120):
    n_features = X.shape[1]

    creator.create("FitnessMax", base.Fitness, weights=(1.0,))
    creator.create("Individual", list, fitness=creator.FitnessMax)

    toolbox = base.Toolbox()
    toolbox.register("attr_float", np.random.uniform, -1, 1)
    toolbox.register("individual", tools.initRepeat,
                     creator.Individual, toolbox.attr_float, n_features)
    toolbox.register("population", tools.initRepeat, list, toolbox.individual)

    toolbox.register("evaluate", evaluate_weights, X=X, y=y)
    toolbox.register("mate", tools.cxBlend, alpha=0.5)
    toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=0.3, indpb=0.2)
    toolbox.register("select", tools.selTournament, tournsize=3)

    pop = toolbox.population(pop_size)
    hof = tools.HallOfFame(1)

    stats = tools.Statistics(lambda ind: ind.fitness.values[0])
    stats.register("avg", np.mean)
    stats.register("max", np.max)
    stats.register("min", np.min)
    stats.register("std", np.std)

    pop, logbook = algorithms.eaSimple(
        pop, toolbox,
        cxpb=0.5, mutpb=0.3,
        ngen=n_generations,
        stats=stats,
        halloffame=hof,
```

```
        verbose=True
    )

    return np.array(hof[0]), logbook
```

In [70]:
```
best_raw, softmax_logbook = run_ga_weight_optimizer(
    X=scaled_df[FEATURE_NAMES].values,
    y=scaled_df['C_anomalum_binary'].values,
    n_generations=80,
    pop_size=100
)

c_anomalum_percent_weights = softmax(best_raw)  # normalized, sum to 1
```

gen	nevals	avg	max	min	std	
0	100	0.11915	0.371533	0	0.0912056	
1	72	0.180496	0.453616	0.0086403	0.103125	
2	67	0.269232	0.453616	0.030241	0.087164	
3	61	0.328806	0.470896	0.133925	0.0715344	
4	59	0.371835	0.518418	0.181446	0.0615564	
5	66	0.415814	0.527058	0.146885	0.0693713	
6	64	0.453097	0.531378	0.28945	0.0393692	
7	70	0.473099	0.544339	0.224648	0.0364611	
8	72	0.488134	0.540019	0.367213	0.0301947	
9	65	0.505457	0.540019	0.462256	0.0191846	
10	58	0.515826	0.544339	0.332651	0.0232004	
11	68	0.521269	0.548659	0.466576	0.0159813	
12	63	0.528829	0.557299	0.436335	0.0155537	
13	65	0.533149	0.561619	0.475216	0.0132748	
14	78	0.536217	0.565939	0.496817	0.0133475	
15	50	0.541012	0.565939	0.440655	0.0198619	
16	66	0.54732	0.57458	0.501137	0.016161	
17	64	0.554923	0.59618	0.475216	0.0165567	
18	59	0.559589	0.59618	0.475216	0.0169475	
19	61	0.564773	0.59618	0.509777	0.0145718	
20	76	0.567927	0.59618	0.496817	0.0159485	
21	60	0.575185	0.59618	0.527058	0.0121889	
22	64	0.57998	0.59618	0.514098	0.012225	
23	62	0.582831	0.59618	0.514098	0.0124176	
24	71	0.583825	0.59618	0.535698	0.0126252	
25	67	0.587756	0.59618	0.557299	0.00970592	
26	66	0.590607	0.59618	0.561619	0.00766251	
27	64	0.591515	0.59618	0.561619	0.00800522	
28	74	0.592033	0.59618	0.552979	0.0075552	
29	74	0.591255	0.59618	0.544339	0.00864073	
30	73	0.591471	0.59618	0.548659	0.00924744	
31	66	0.592552	0.59618	0.57026	0.00690467	
32	61	0.592681	0.600501	0.561619	0.00784068	
33	72	0.593156	0.600501	0.552979	0.00716416	
34	72	0.592379	0.600501	0.561619	0.00723622	
35	75	0.594193	0.600501	0.57026	0.00516325	
36	55	0.594107	0.600501	0.57026	0.00633399	
37	65	0.593632	0.600501	0.540019	0.00908454	
38	59	0.593675	0.600501	0.561619	0.00883637	
39	65	0.595921	0.600501	0.557299	0.00704118	
40	61	0.596051	0.600501	0.565939	0.00799998	
41	58	0.59752	0.600501	0.565939	0.00606962	
42	61	0.59618	0.600501	0.565939	0.00796596	
43	77	0.595792	0.600501	0.57026	0.00753479	
44	67	0.596656	0.600501	0.544339	0.00791649	
45	60	0.596958	0.600501	0.565939	0.00729018	
46	59	0.598254	0.600501	0.565939	0.00534303	
47	73	0.595792	0.600501	0.557299	0.00846782	
48	60	0.597001	0.600501	0.57026	0.00729824	
49	76	0.597952	0.600501	0.565939	0.00583124	
50	74	0.59631	0.600501	0.548659	0.00978406	
51	63	0.596569	0.600501	0.557299	0.0081075	
52	70	0.59752	0.600501	0.565939	0.00651455	
53	71	0.596396	0.600501	0.552979	0.00881935	
54	63	0.598081	0.600501	0.565939	0.00577415	

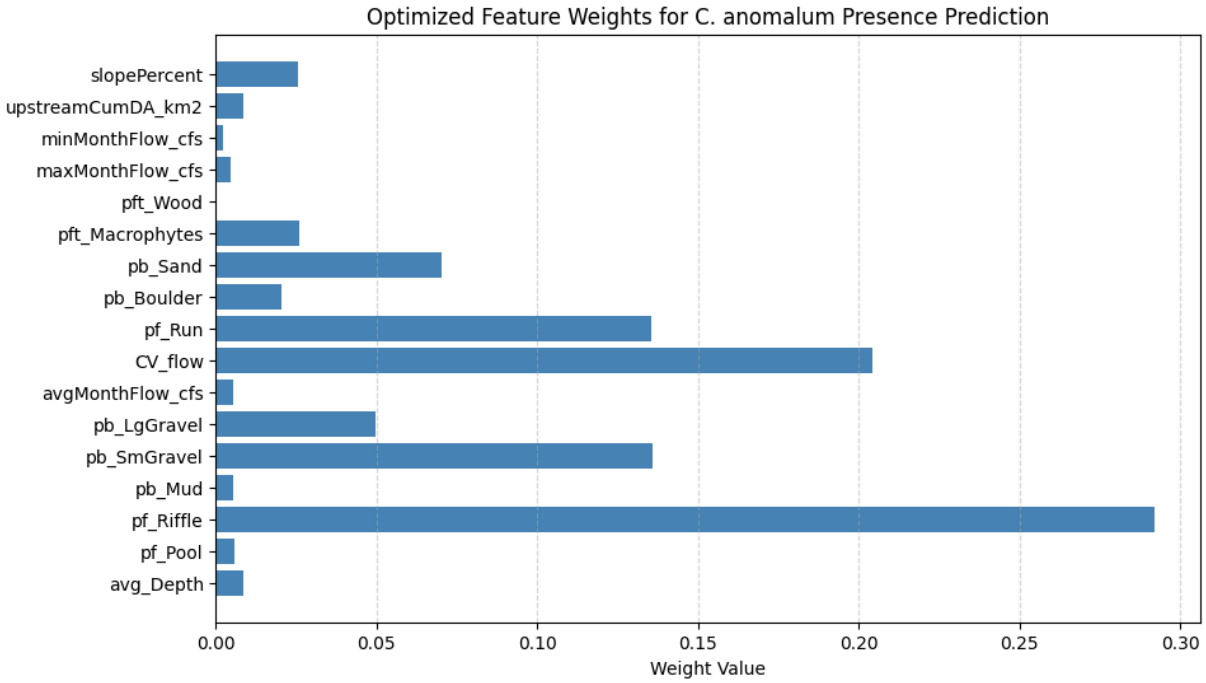
55	60	0.5986	0.600501	0.5789	0.00493784
56	71	0.59726	0.600501	0.57026	0.00705587
57	69	0.596483	0.604821	0.557299	0.00817992
58	74	0.597304	0.600501	0.57026	0.00684767
59	64	0.596915	0.600501	0.565939	0.00707384
60	58	0.597001	0.600501	0.565939	0.00769654
61	59	0.599075	0.600501	0.57026	0.00440803
62	70	0.596828	0.600501	0.522738	0.00993399
63	57	0.597001	0.604821	0.561619	0.00784068
64	61	0.597693	0.604821	0.552979	0.0076156
65	66	0.597001	0.604821	0.565939	0.00719522
66	71	0.597131	0.604821	0.57458	0.00639557
67	71	0.597304	0.604821	0.561619	0.00656947
68	67	0.597779	0.604821	0.57026	0.00568736
69	61	0.598038	0.604821	0.561619	0.00647316
70	67	0.599032	0.604821	0.58322	0.00428632
71	62	0.598211	0.604821	0.565939	0.00596916
72	62	0.59739	0.604821	0.552979	0.00770587
73	69	0.597563	0.604821	0.540019	0.00885114
74	64	0.598729	0.604821	0.557299	0.0074099
75	70	0.599248	0.604821	0.57026	0.00708017
76	61	0.59968	0.604821	0.561619	0.00863371
77	63	0.600112	0.604821	0.57458	0.00699531
78	62	0.600587	0.604821	0.561619	0.00743216
79	70	0.600846	0.604821	0.58322	0.00581794
80	58	0.599377	0.604821	0.509777	0.012106

In [71]:
```
for name, w in zip(FEATURE_NAMES, c_anomalum_percent_weights):
    print(f"{name}: {w*100:.4f}%")

print("Sum of weights:", c_anomalum_percent_weights.sum())
```

```
avg_Depth: 0.8486%
pf_Pool: 0.5878%
pf_Riffle: 29.1827%
pb_Mud: 0.5499%
pb_SmGravel: 13.5878%
pb_LgGravel: 4.9631%
avgMonthFlow_cfs: 0.5422%
CV_flow: 20.4309%
pf_Run: 13.5382%
pb_Boulder: 2.0239%
pb_Sand: 7.0276%
pft_Macrophytes: 2.5800%
pft_Wood: 0.0284%
maxMonthFlow_cfs: 0.4439%
minMonthFlow_cfs: 0.2355%
upstreamCumDA_km2: 0.8666%
slopePercent: 2.5631%
Sum of weights: 1.0
```

In [72]:
```
plt.figure(figsize=(10,6))
plt.barh(FEATURE_NAMES, c_anomalum_percent_weights, color="steelblue")
plt.xlabel("Weight Value")
plt.title("Optimized Feature Weights for C. anomalum Presence Prediction")
```

```
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.show()
```



Normalized, Ecologically Interpretable GA Weights

> To make the genetic algorithm's feature weights ecologically interpretable, we treat them as a probability distribution over environmental metrics. Rather than using raw signed coefficients, we evolve unconstrained weights and then transform them with a softmax function inside the fitness evaluation. This ensures that all weights are non-negative and sum to 1, so each weight can be interpreted as the relative importance of that metric for predicting *Campostoma anomalum* presence. The GA then optimizes this normalized weight vector to maximize the Spearman correlation between the weighted combination of metrics and the binary presence data.

> Now I will create a vector that is our "suitability score" created by our best weights.

> We will use this later for our prediction models.

In [73]:
```
c_anomalum_suit = np.dot(scaled_df[FEATURE_NAMES].values, c_anomalum_weights)

print("Suitability Scores (first 10):", c_anomalum_suit[:10])
print(type(c_anomalum_suit))
```

```
Suitability Scores (first 10): [-0.35646125 -0.01890942  0.85282377 -1.0725474   0.3
5524341  0.28058386
 -1.23859547  0.53571254 -0.69292343 -0.23373791]
<class 'numpy.ndarray'>
```

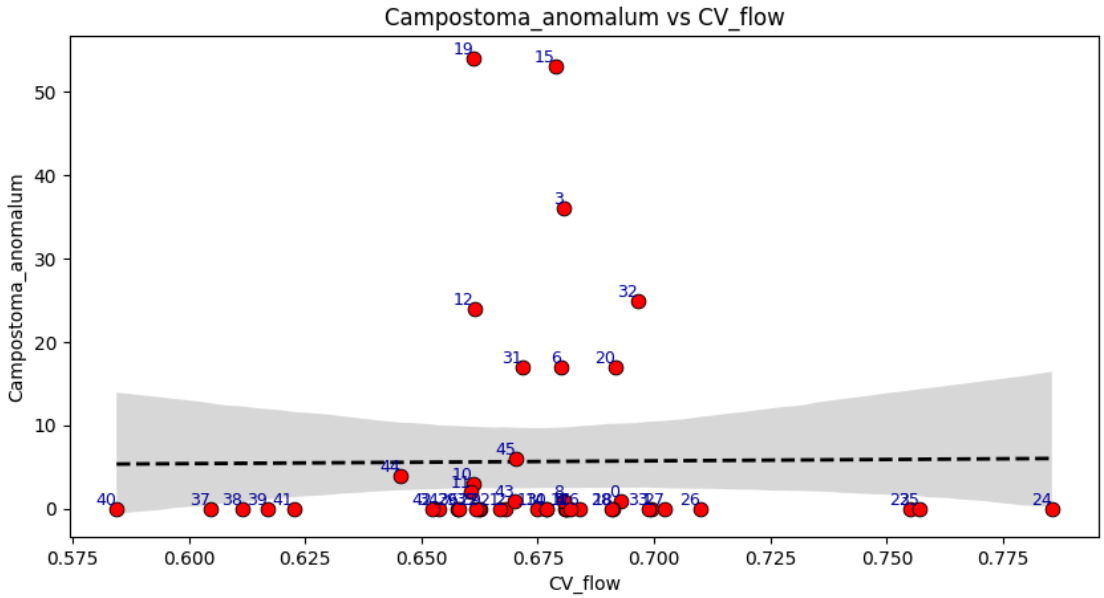
Feature Trend Line Fit

> Starting with our C. Anomalum's population relation with percent bottom.

In [74]:
```
scatter_trendLine_plotter(s_ed['CV_flow'], s_fcd['Campostoma_anomalum'])
```

functions.py

Korbin Brashears
07/25/2026

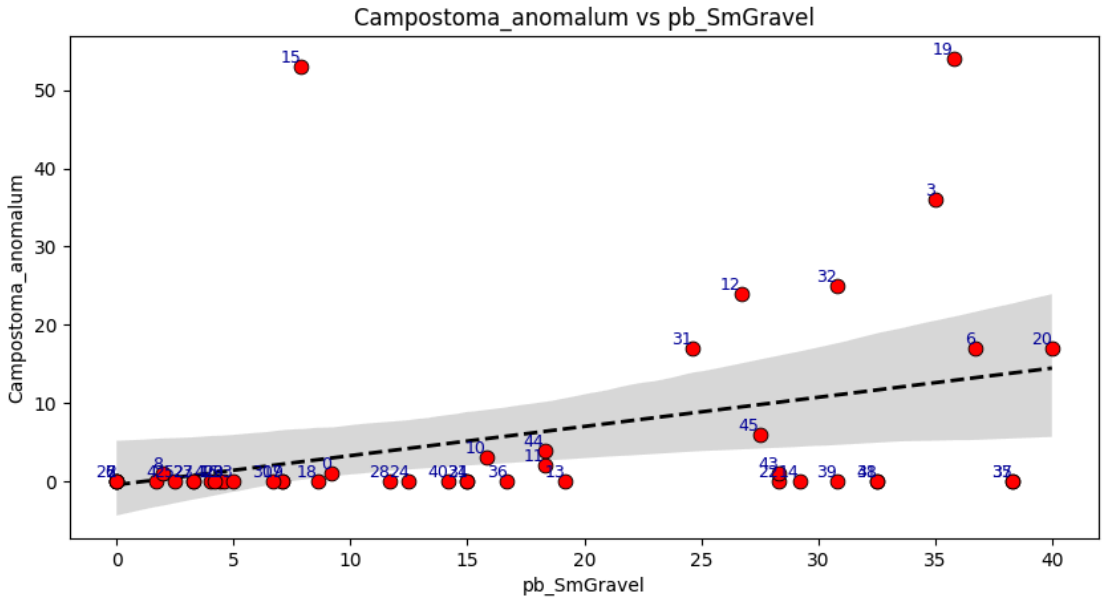


seth_envData_2025.csv

In [75]:
```
scatter_trendLine_plotter(s_ed['pb_SmGravel'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(s_ed['pb_LgGravel'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(s_ed['pb_Cobble'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(s_ed['pb_Mud'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(s_ed['pb_Sand'], s_fcd['Campostoma_anomalum'])
```

functions.py

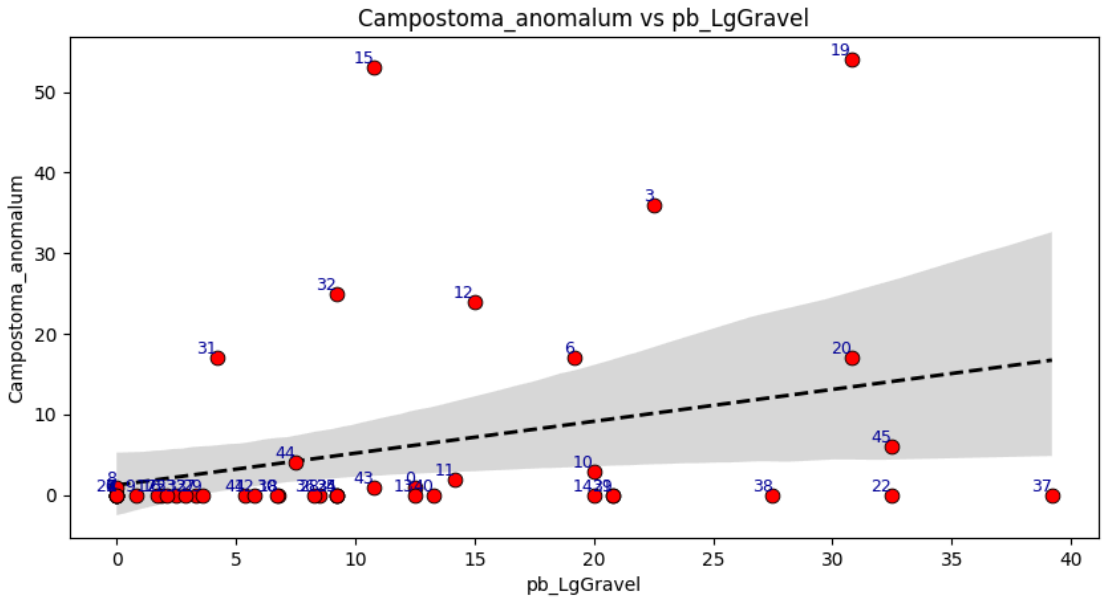
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

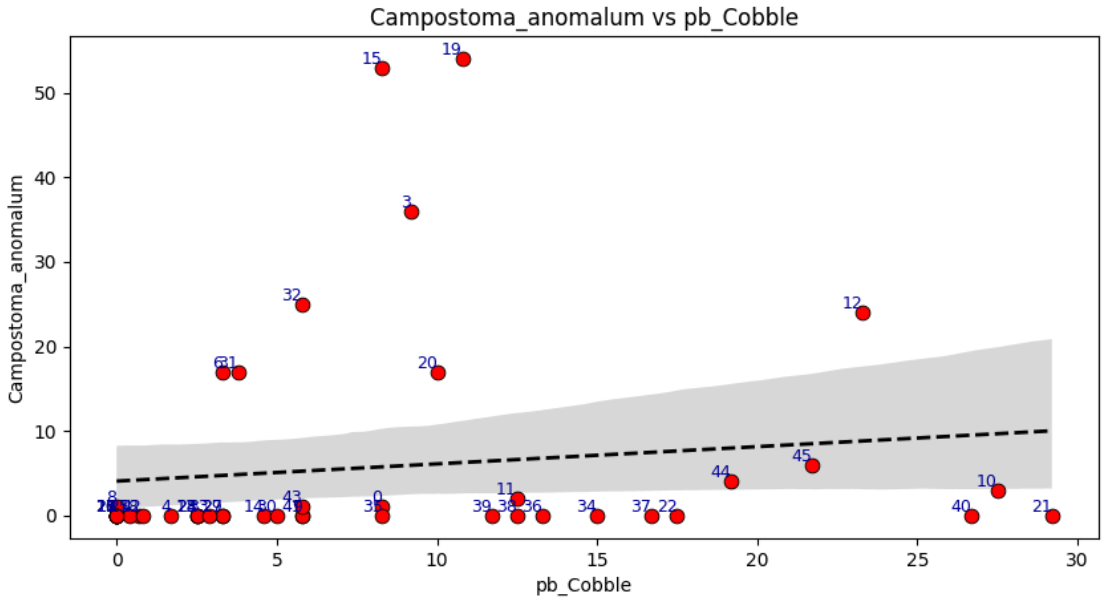
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

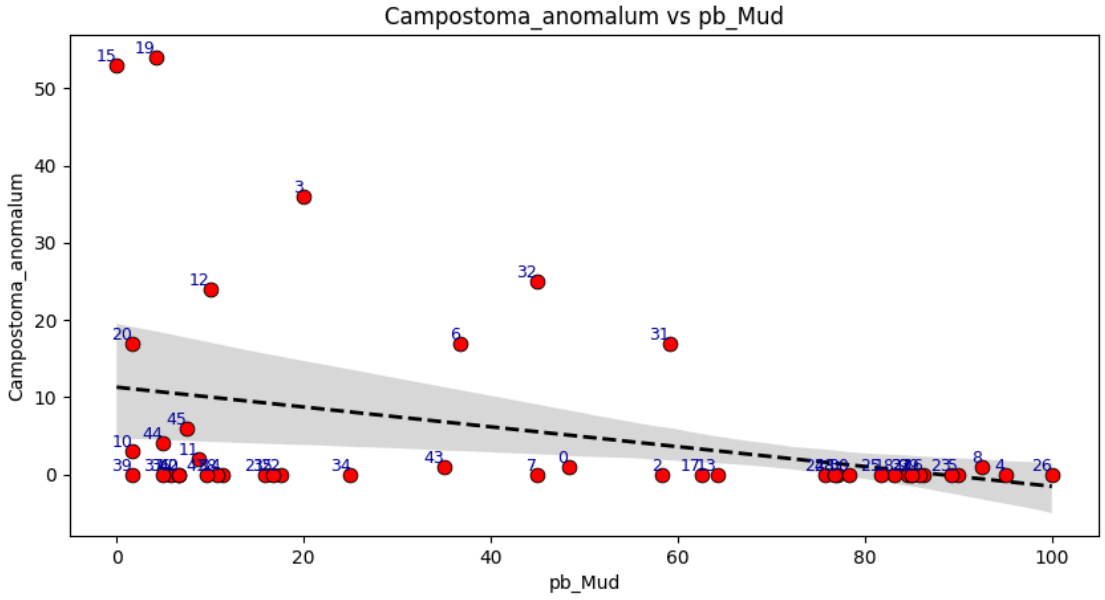
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

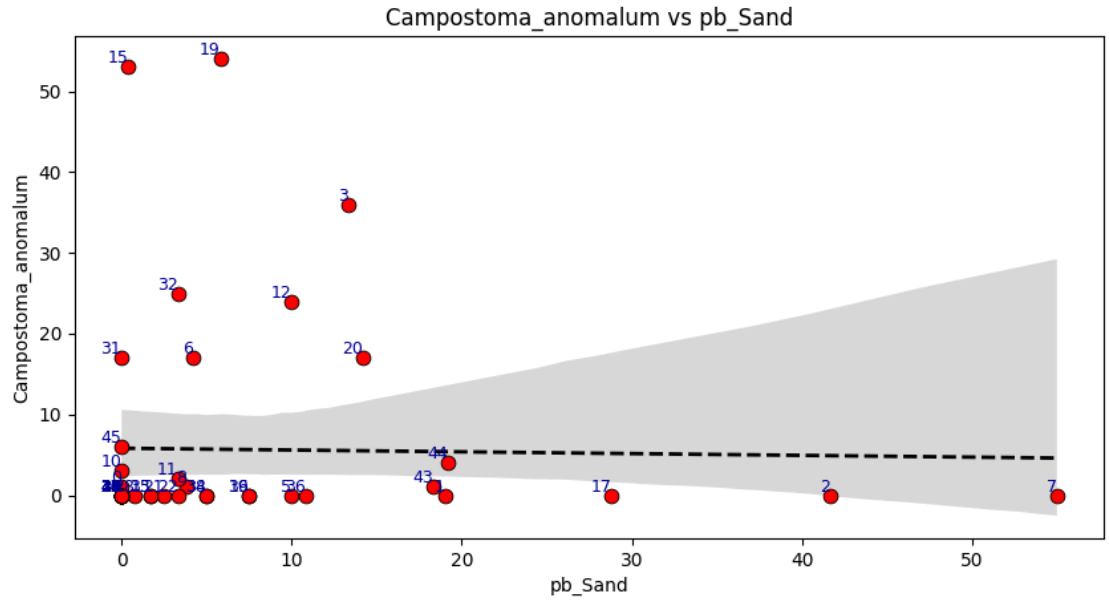
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

Korbin Brashears
07/25/2026



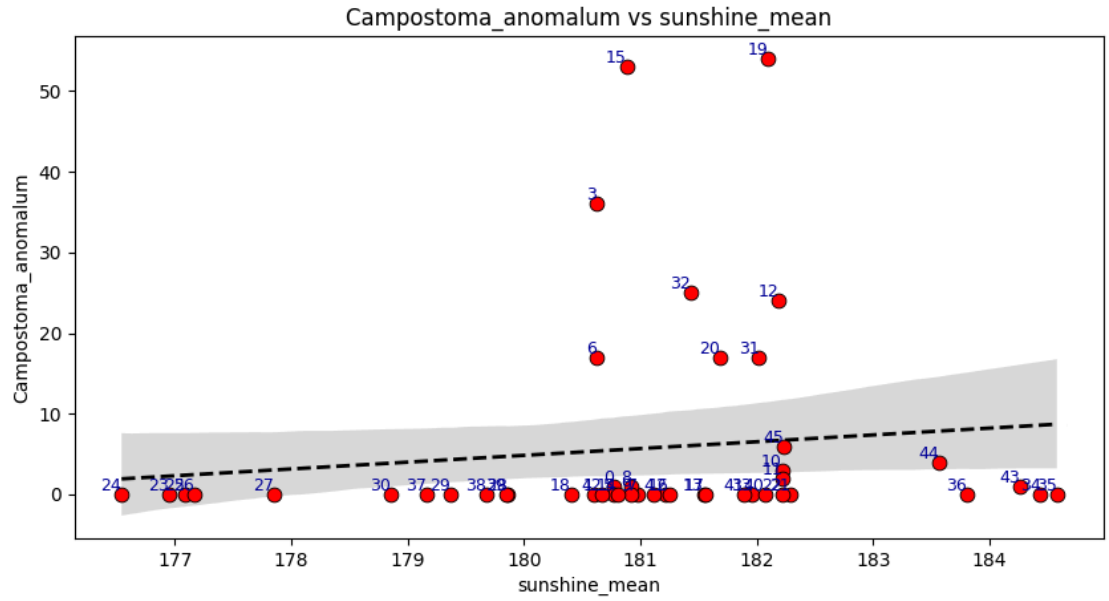
seth_envData_2025.csv

> Next we will look at C. Anomalum's population relation with site_N.sunshine.

In [76]:
```
scatter_trendLine_plotter(g_10dts_m['sunshine_mean'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(g_10dts_m['sunshine_max'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(g_10dts_m['sunshine_min'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(g_10dts_m['sunshine_sd'], s_fcd['Campostoma_anomalum'])

scatter_trendLine_plotter(g_10dts_m['sunshine_trend'], s_fcd['Campostoma_anomalum']
```

functions.py

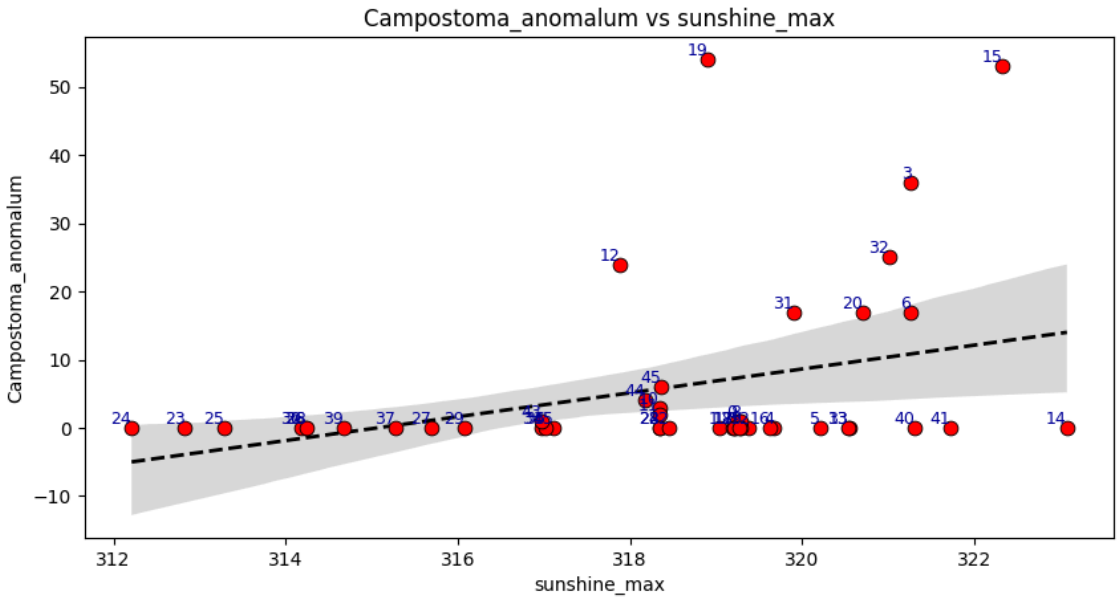
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

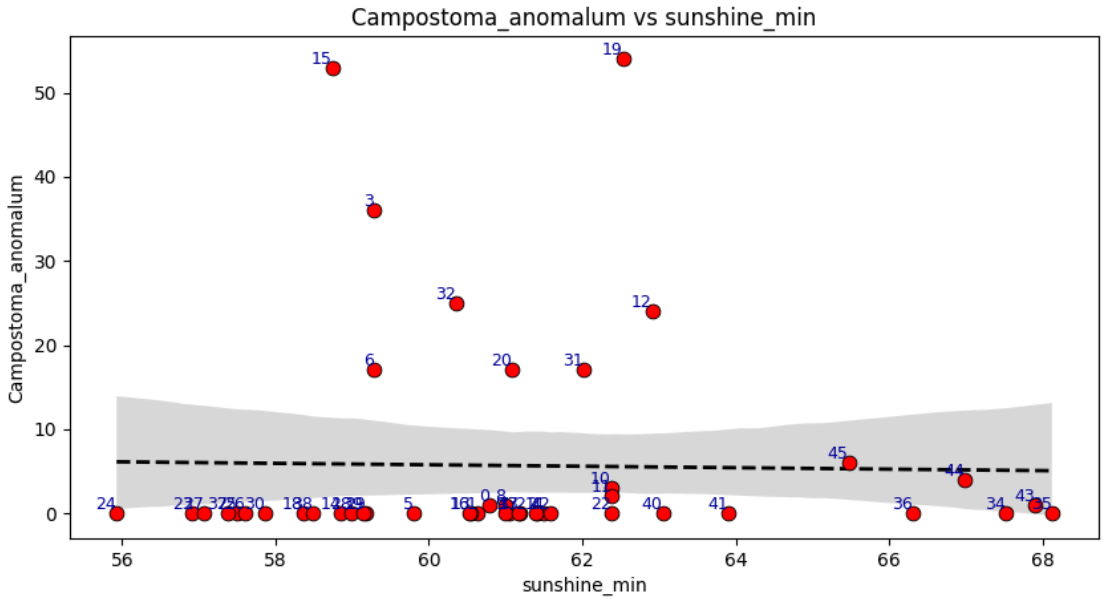
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

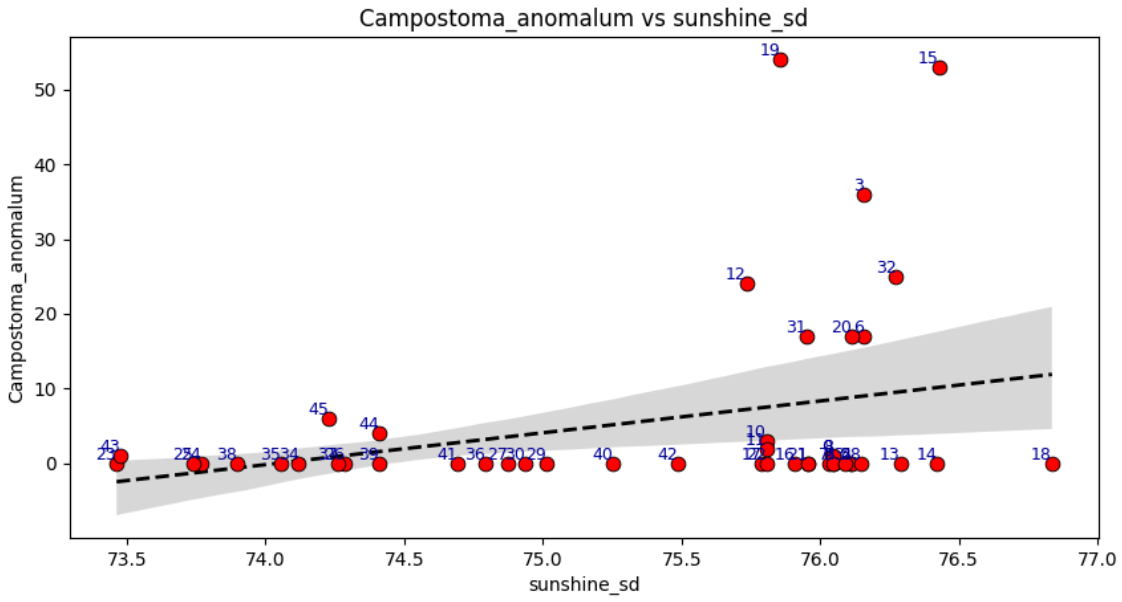
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

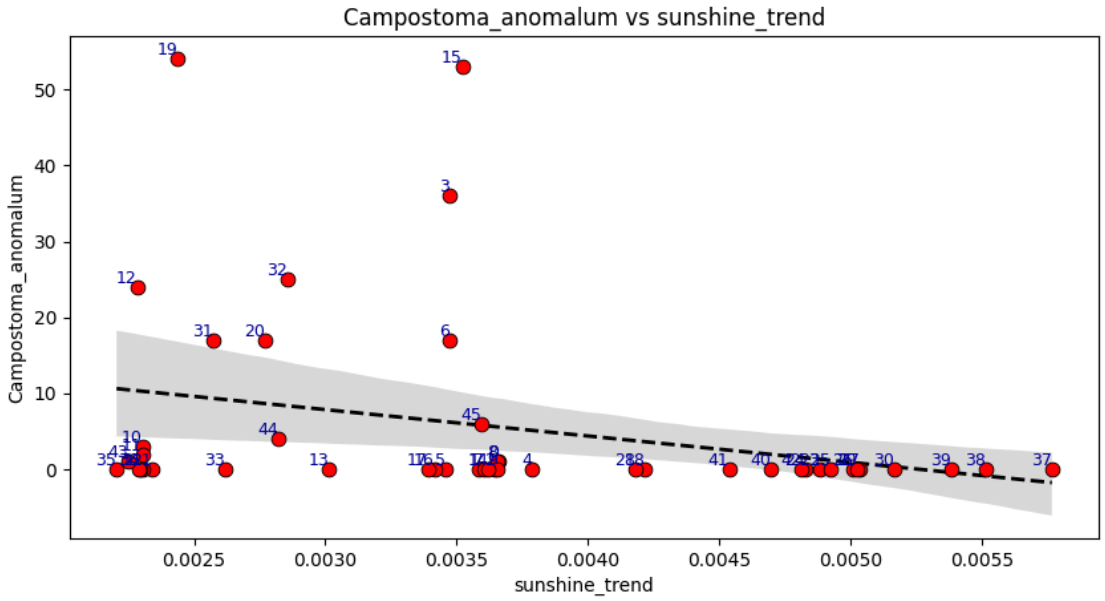
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

Korbin Brashears
07/25/2026



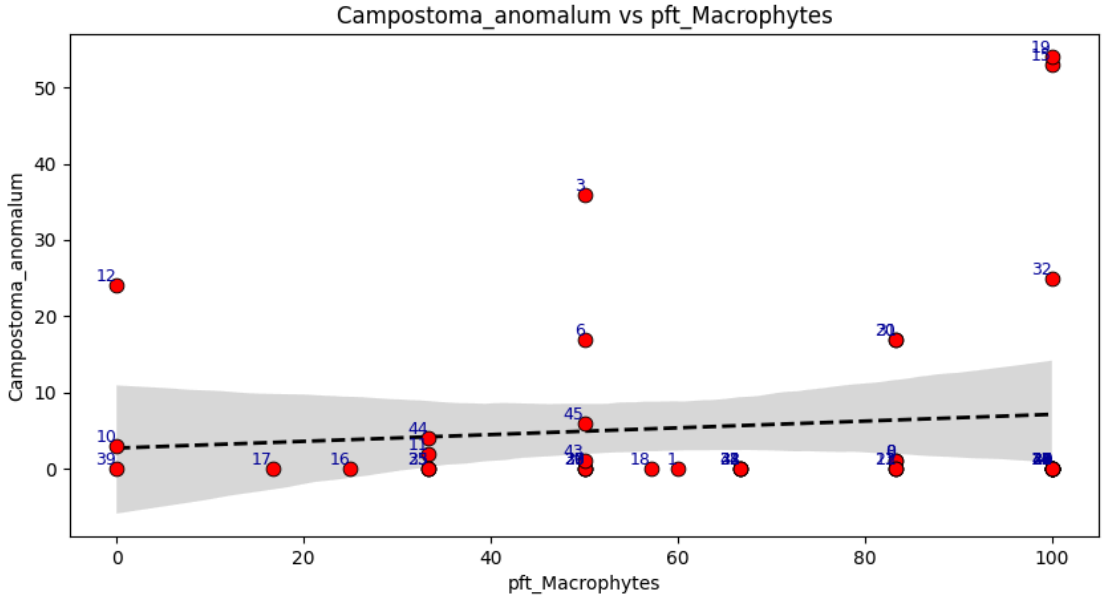
seth_envData_2025.csv

> Next we will look at C. Anomalum's population relation with pft_Macrophytes and pft_Wood.

In [77]:
```
scatter_trendLine_plotter(s_ed['pft_Macrophytes'], s_fcd['Campostoma_anomalum'])
scatter_trendLine_plotter(s_ed['pft_Wood'], s_fcd['Campostoma_anomalum'])
```

functions.py

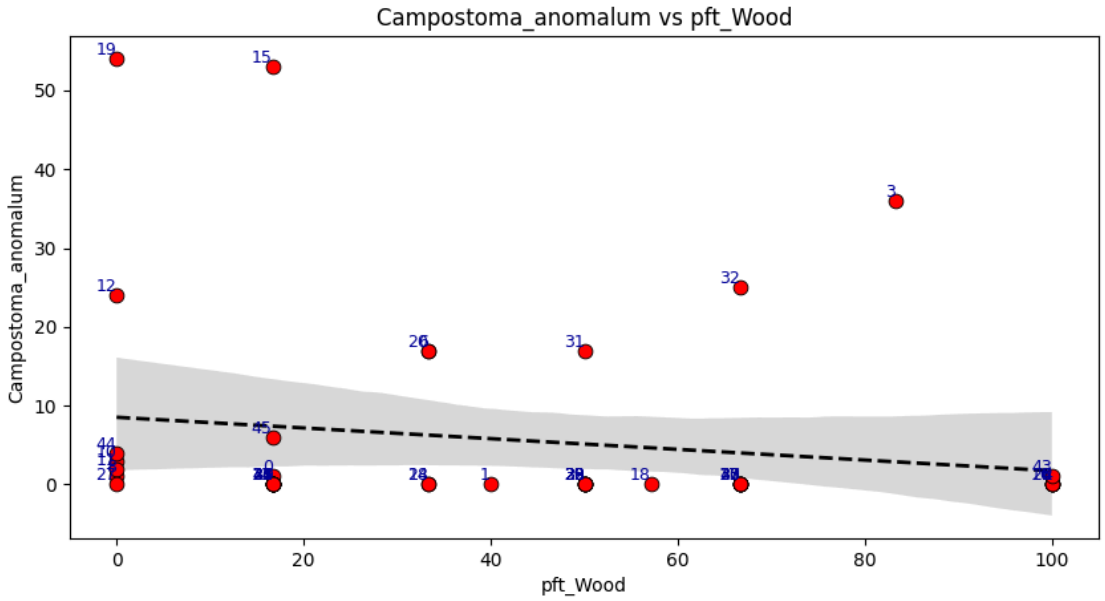
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

Korbin Brashears
07/25/2026



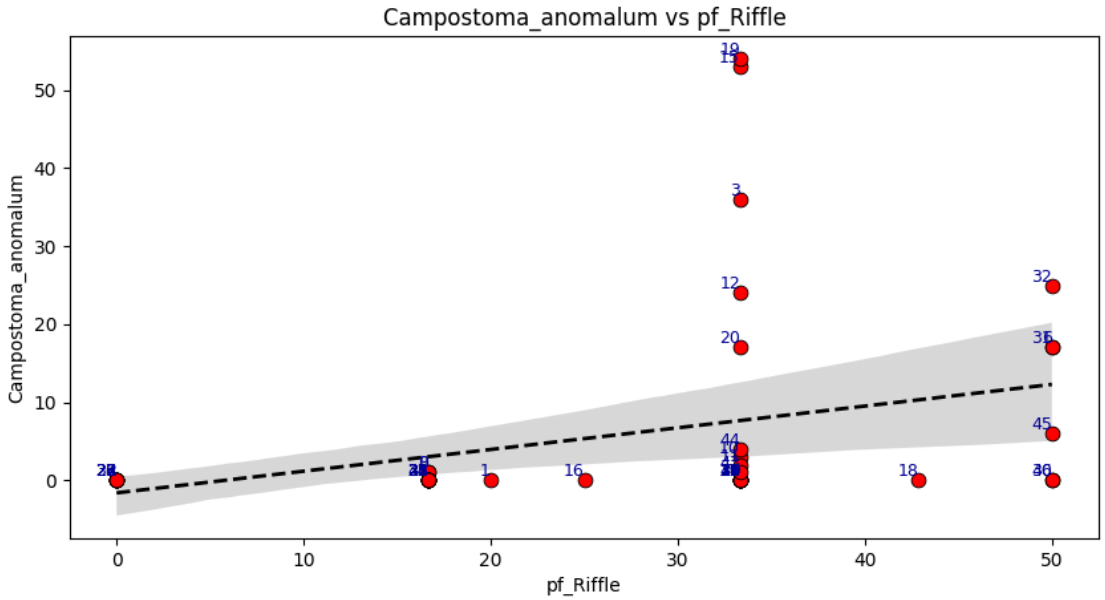
seth_envData_2025.csv

> Next we will look at C. Anomalum's population relation with the percent flow metrics (pf_Riffle, pf_Run, pf_Pool).

In [78]:
```
scatter_trendLine_plotter(s_ed['pf_Riffle'], s_fcd['Campostoma_anomalum'])
scatter_trendLine_plotter(s_ed['pf_Run'], s_fcd['Campostoma_anomalum'])
scatter_trendLine_plotter(s_ed['pf_Pool'], s_fcd['Campostoma_anomalum'])
```

functions.py

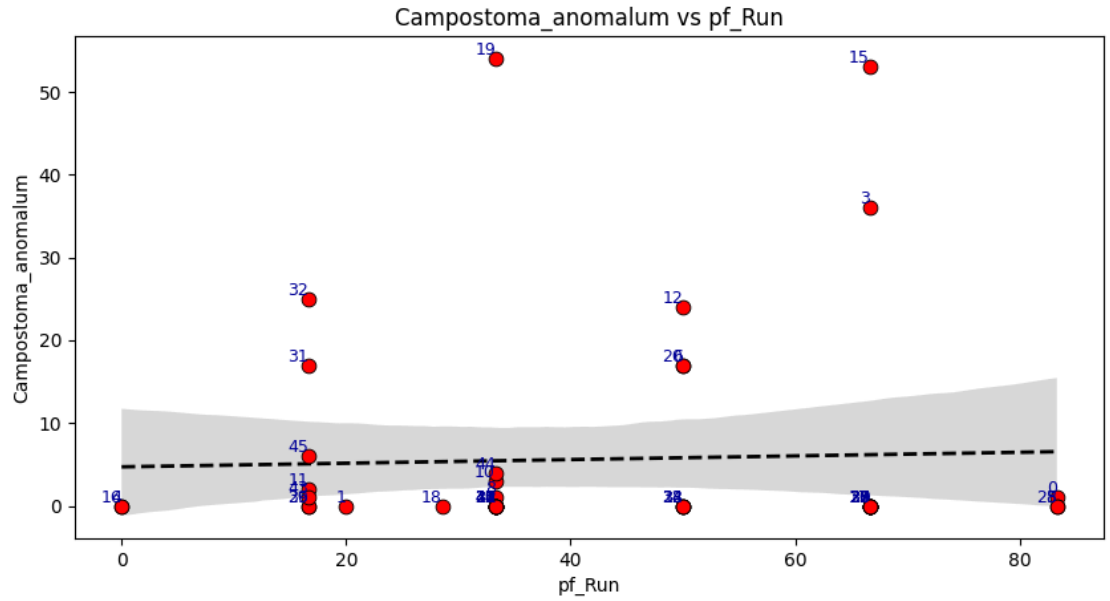
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

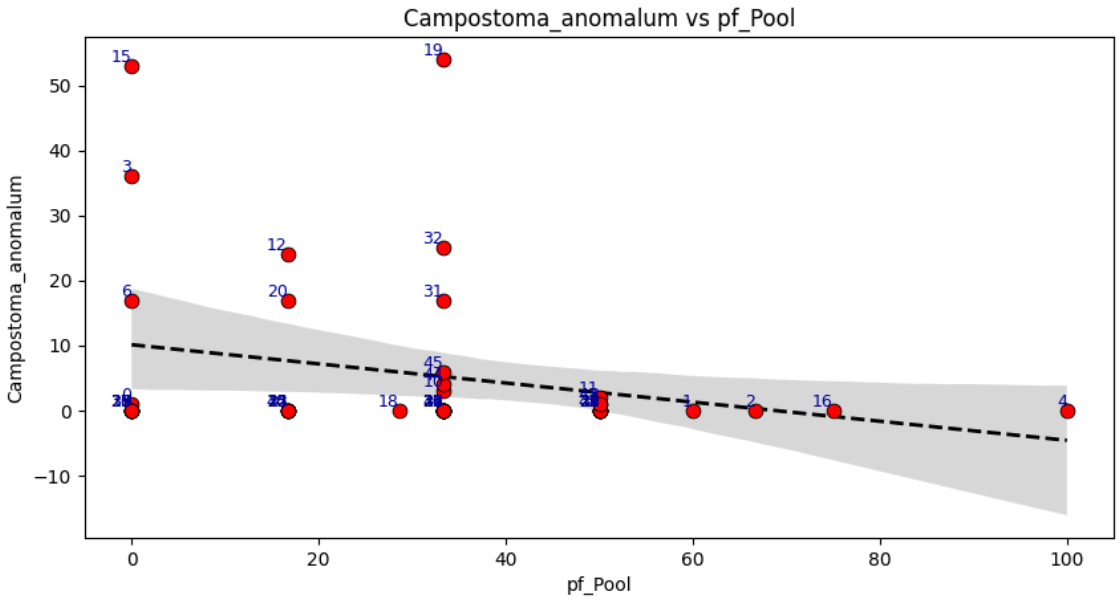
Korbin Brashears
07/25/2026



seth_envData_2025.csv

functions.py

Korbin Brashears
07/25/2026



seth_envData_2025.csv

Ensemble and Classifier Models

> Now we will use these metrics to see

Creating Our Models

In [79]:
```
df = scaled_df[['C_anomalum_count', 'C_anomalum_binary']].copy()
df['suitability'] = c_anomalum_suit

df.head()
```

Out[79]:

	C_anomalum_count	C_anomalum_binary	suitability
0	1.0	0	-0.356461
1	0.0	0	-0.018909
2	0.0	0	0.852824
3	36.0	1	-1.072547
4	0.0	0	0.355243

In [80]:
```
X = df['suitability']
y = df['C_anomalum_binary']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

model = XGBClassifier(
        n_estimators=500,
        learning_rate=0.05,
        max_depth=6,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="binary:logistic"
    )
model.fit(X_train, y_train)
y_pred_prob = model.predict_proba(X_test)[:, 1]
```

In [81]:
```
from sklearn.metrics import roc_auc_score, accuracy_score, f1_score

y_pred = (y_pred_prob >= 0.5).astype(int)
auc = roc_auc_score(y_test, y_pred_prob)
acc = accuracy_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print(f"auc: {auc}")
print(f"acc: {acc}")
print(f"f1: {f1}")
```

```
auc: 0.90625
acc: 0.9
f1: 0.8
```

In [82]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# ------------------------------
# 1. Select predictors & target
```

```
# ------------------------------
X = df['suitability']        # or df[predictor_columns]
y = df['C_anomalum_count']   # abundance instead of binary

# ------------------------------
# 2. Train/test split
# ------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# XGBRegressor expects 2D input
X_train = np.array(X_train).reshape(-1, 1)
X_test  = np.array(X_test).reshape(-1, 1)

# ------------------------------
# 3. Train the model
# ------------------------------
model = XGBRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    objective="reg:squarederror"
)

model.fit(X_train, y_train)

# ------------------------------
# 4. Predict
# ------------------------------
y_pred = model.predict(X_test)

# ------------------------------
# 5. Evaluate
# ------------------------------
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"RMSE: {rmse}")
print(f"MAE:  {mae}")
print(f"R²:   {r2}")
```

```
RMSE: 19.14384441052738
MAE:  9.798444532824215
R²:   -1.4990574757209703
```